
SOFTWARE ARCHITECTURE DOCUMENT (SAD)

for

LKFORUM WEBSITE

Version 1.1

Prepared by : Quach Gia Kiet

Submitted to : Nguyen Trinh Dong
Lecturer

May 12, 2026

Contents

1 Documentation Roadmap

1.1 Document Management and Configuration Control Information

This document serves as the Software Architecture Document (SAD) for the LKForum project. It provides a comprehensive architectural overview of the system, using a number of different architectural views to depict different aspects of the system.

1.2 Purpose and Scope of the SAD

The purpose of this document is to define and describe the software architecture of the LKForum platform, a community-driven social networking site. The scope includes the backend API (Go/Gin), the frontend SPAs (Svelte), the data storage strategy (MongoDB, Redis), and the deployment environment (Docker). This document is intended to be read by developers, system administrators, and project stakeholders.

1.3 How the SAD Is Organized

This SAD follows the SEI "Views and Beyond" documentation template.

- **Chapter 1:** Documentation Roadmap.
- **Chapter 2:** Architecture Background (Problem and Solution contexts).
- **Chapter 3:** Views (System Context, Module, and Allocation views).
- **Chapter 4:** Relations Among Views.

2 Architecture Background

2.1 Problem Background

2.1.1 System Overview

LKForum is a modern community social networking platform designed to facilitate user engagement, content sharing, and real-time communication. It provides isolated sub-forums (communities) where users can post content, vote, comment, and engage in real-time private messaging.

2.1.2 Significant Driving Requirements

The software architecture was driven by several key Quality Attributes (documented deeply in the Attribute-Driven Design - ADD document):

- **Performance:** High-throughput feed retrieval and real-time WebSocket messaging.
- **Security:** Strong authentication (JWT), RBAC, and End-to-End Encryption (E2EE) for messages.
- **Maintainability:** Strict Layered Architecture to facilitate future feature expansions.

2.2 Solution Background

2.2.1 Architectural Approaches

The solution utilizes a **Client-Server** architectural style combined with an **Event-Driven** internal approach.

- **Backend:** A monolithic Go application utilizing the Gin framework for high-performance HTTP routing. It employs a Layered Architecture (Controller → Service → Repository) to isolate business logic.
- **Frontend:** Separate Single Page Applications (SPAs) built with Svelte for distinct administrative and user boundaries.
- **Asynchronous Processing:** Go Channels are used as an in-memory Event Bus to handle tasks like notifications without blocking main threads.
- **Persistence & Caching:** MongoDB provides flexible schema design for varying post types, while Redis serves as a Read-Through cache and session store.

3 Views

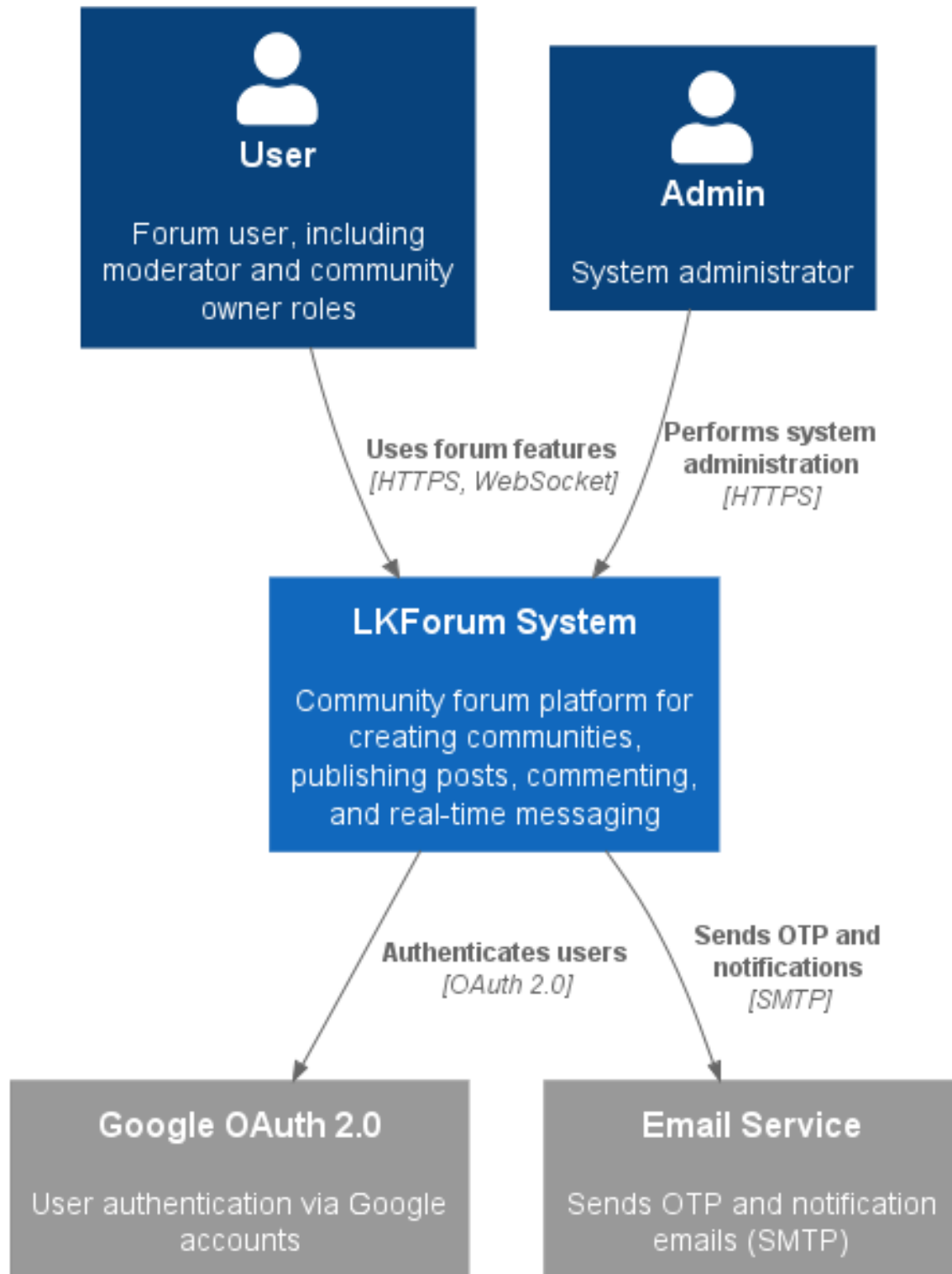
This section specifies the software architecture using multiple perspectives based on the C4 Model and SEI Viewpoints. Each view is accompanied by an Element Catalog that provides deeper textual descriptions of the components.

3.1 Component-and-Connector View (System Context)

This view illustrates how the system interacts with its environment and external actors at runtime.

3.1.1 Primary Presentation

System Context Diagram - LKForum



Legend

	person
	system
	external system

3.1.2 Element Catalog

- **Platform User (Actor):** Represents any individual interacting with the LKForum platform. This includes Guests browsing public content, authenticated Members posting content, and Moderators managing communities.
- **System Administrator (Actor):** A specialized user responsible for global platform health, handling site-wide reports, and configuring overarching system parameters.
- **LKForum System (Software System):** The core subject of this architecture. It processes all business logic, serves user interfaces, and maintains the primary state of the social network.
- **Google OAuth (External System):** A third-party Identity Provider used to facilitate Single Sign-On (SSO), allowing users to authenticate without creating local passwords.
- **Cloudinary (External System):** A third-party media management service. It offloads the burden of storing, resizing, and serving static media assets (avatars, post images) from the primary backend.
- **Email Provider (External System):** An SMTP gateway (e.g., SendGrid or AWS SES) used for delivering transactional emails such as OTPs and password resets.

3.1.3 Behavior: Sequence Diagrams

This section contains sequence diagrams that describe the exact runtime behavior and interactions between internal and external components for critical use cases.

UC-01: Sign Up

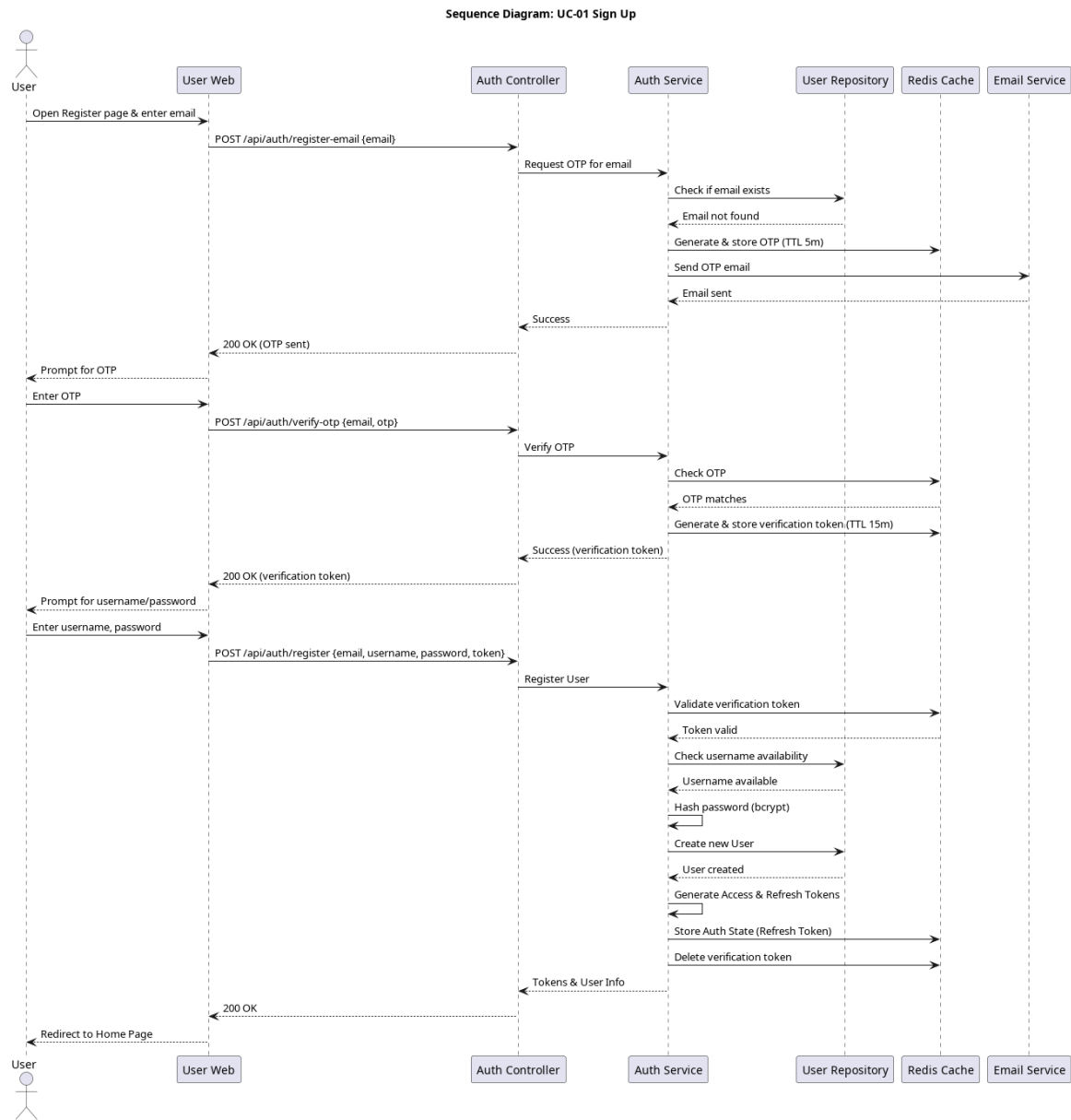


Figure 3.2: Sequence Diagram for Sign Up

UC-02.1: Local Sign In

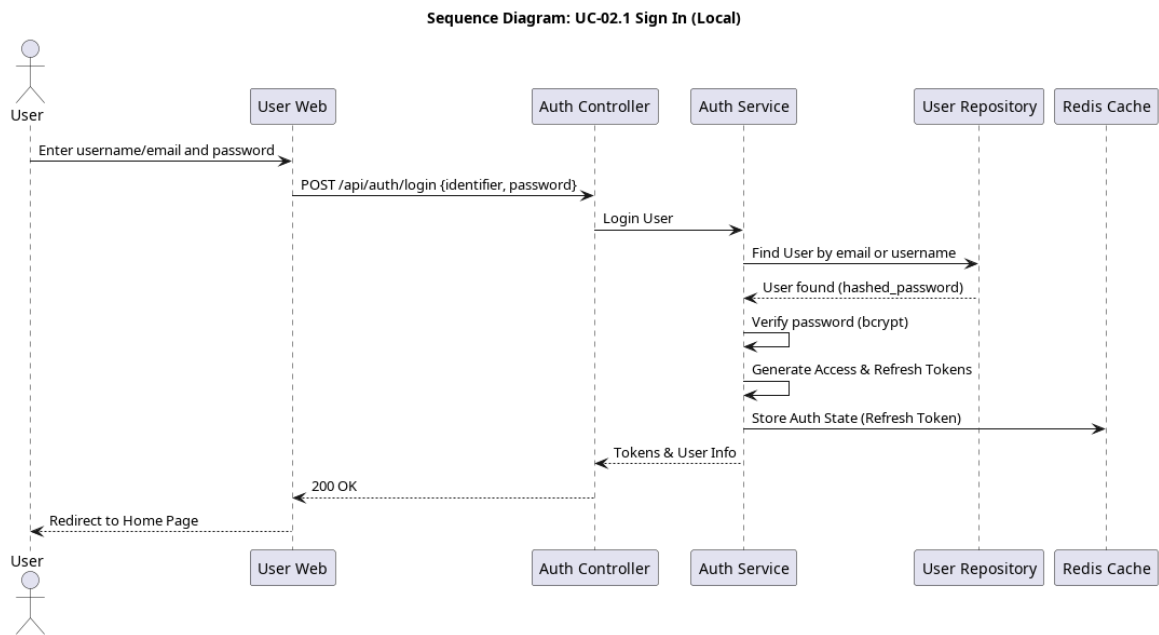


Figure 3.3: Sequence Diagram for Local Sign In

UC-02.2: Google Sign In

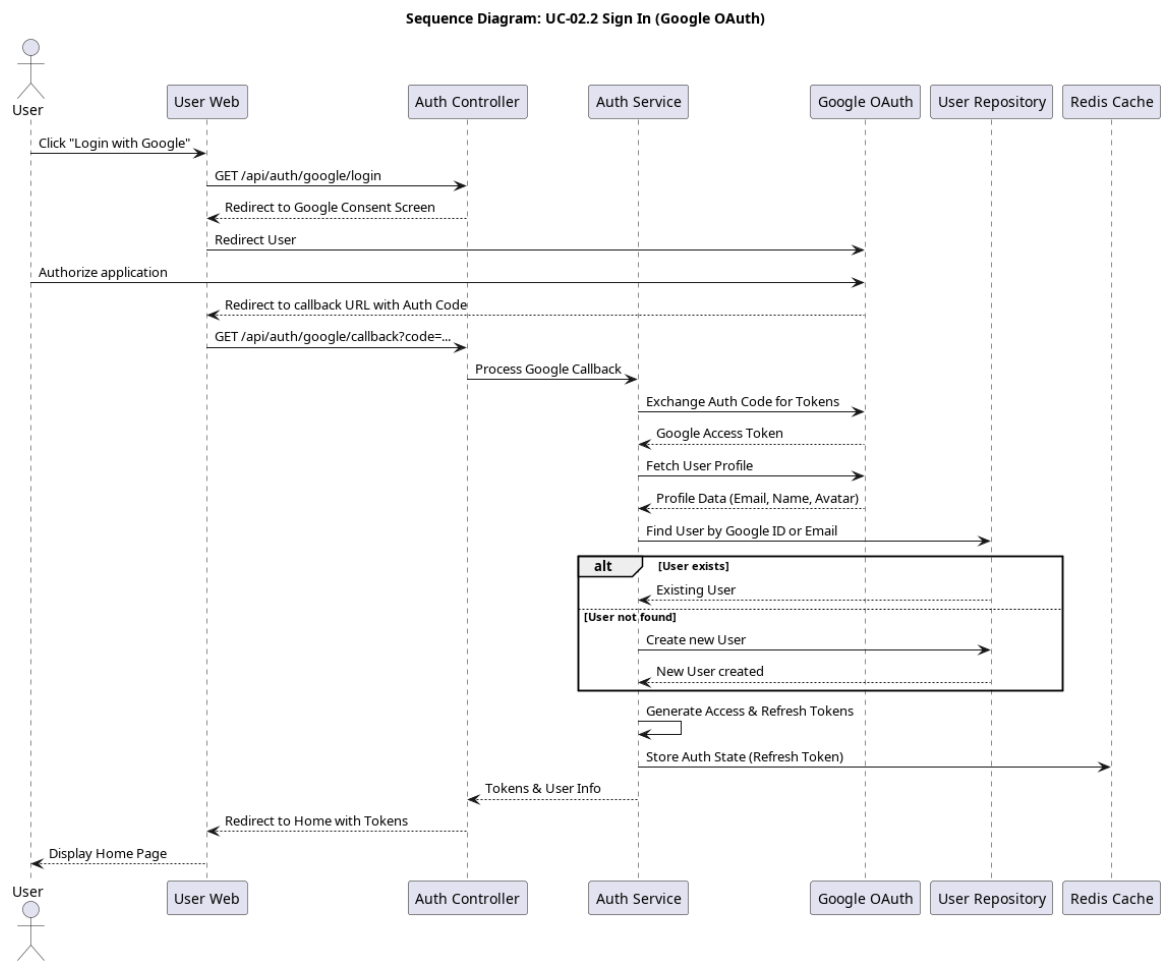


Figure 3.4: Sequence Diagram for Google OAuth Sign In

UC-03: Forgot Password

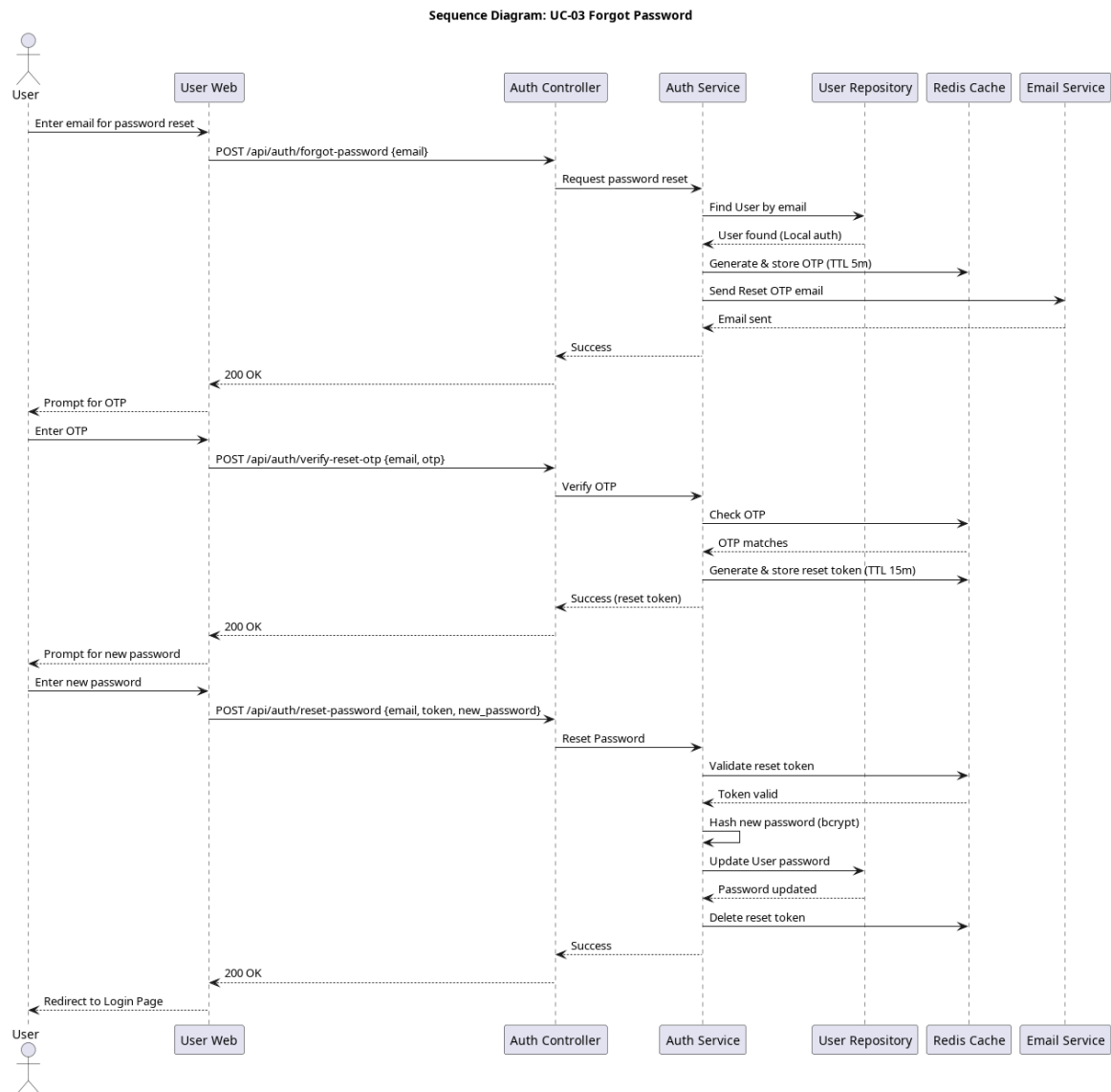


Figure 3.5: Sequence Diagram for Forgot Password

UC-04: Manage Profile

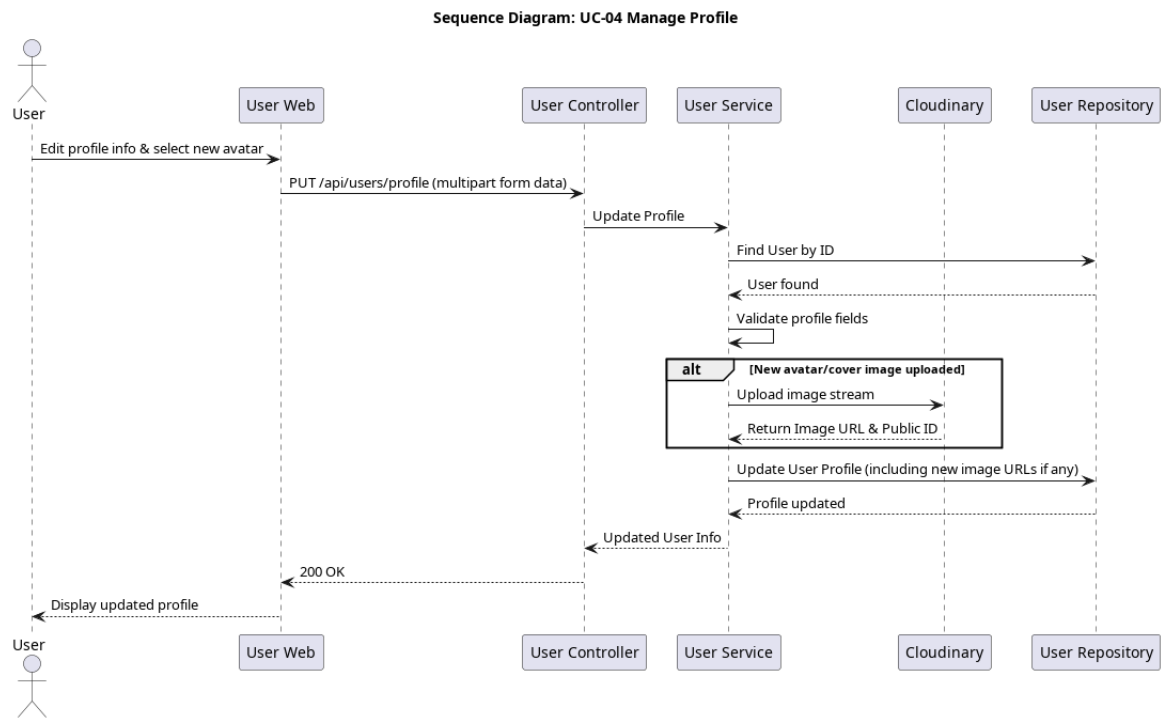


Figure 3.6: Sequence Diagram for Manage Profile

UC-05: Change Settings

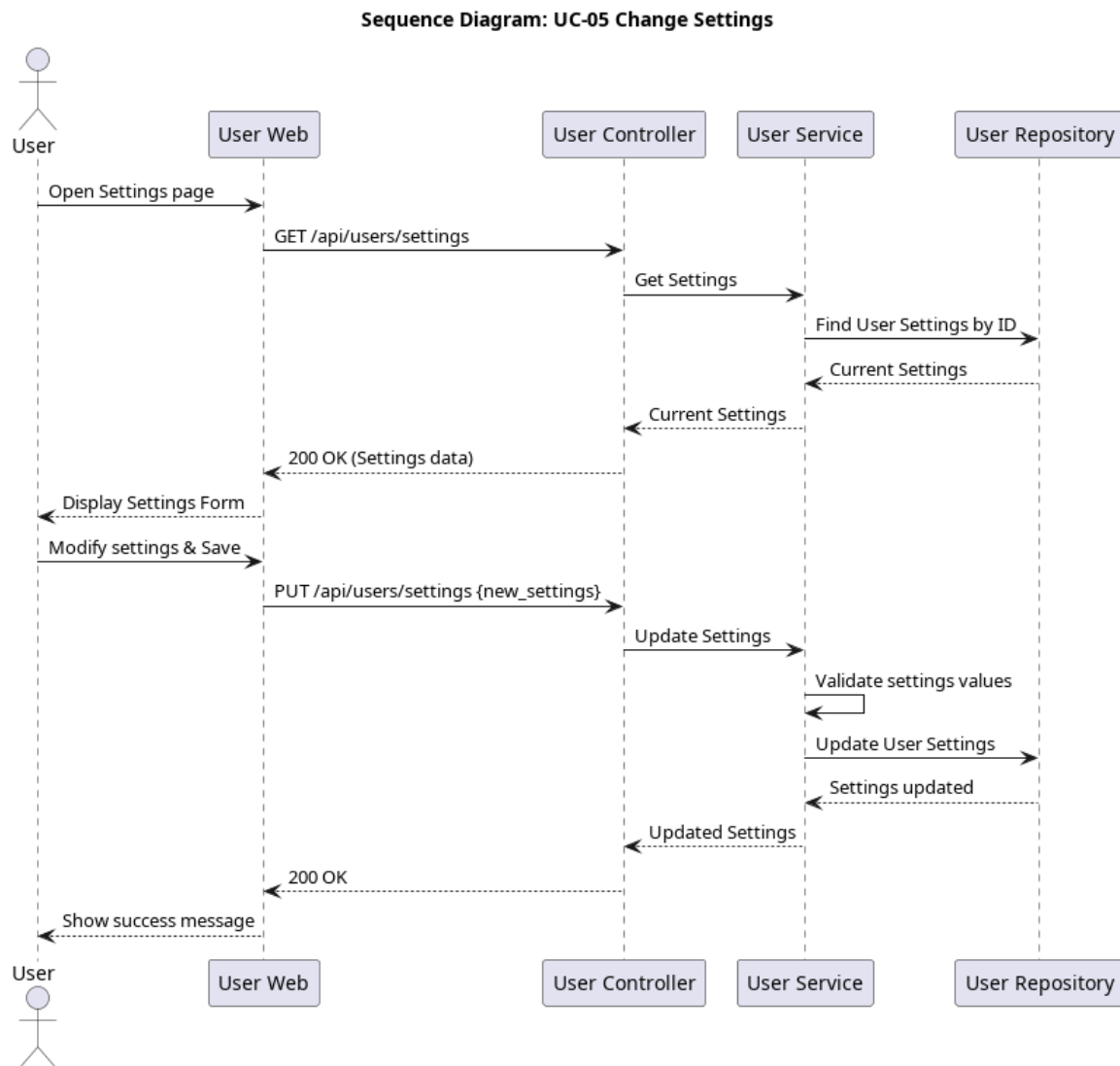


Figure 3.7: Sequence Diagram for Change Settings

UC-06: View Activity History

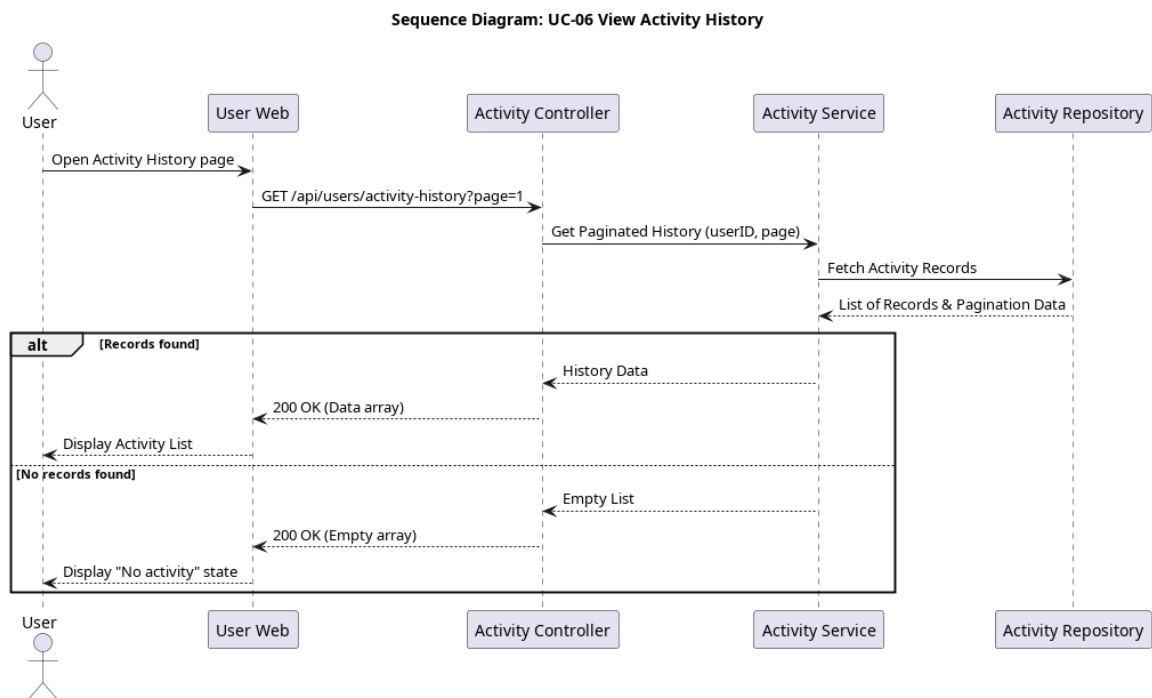


Figure 3.8: Sequence Diagram for View Activity History

UC-07: Follow / Unfollow User

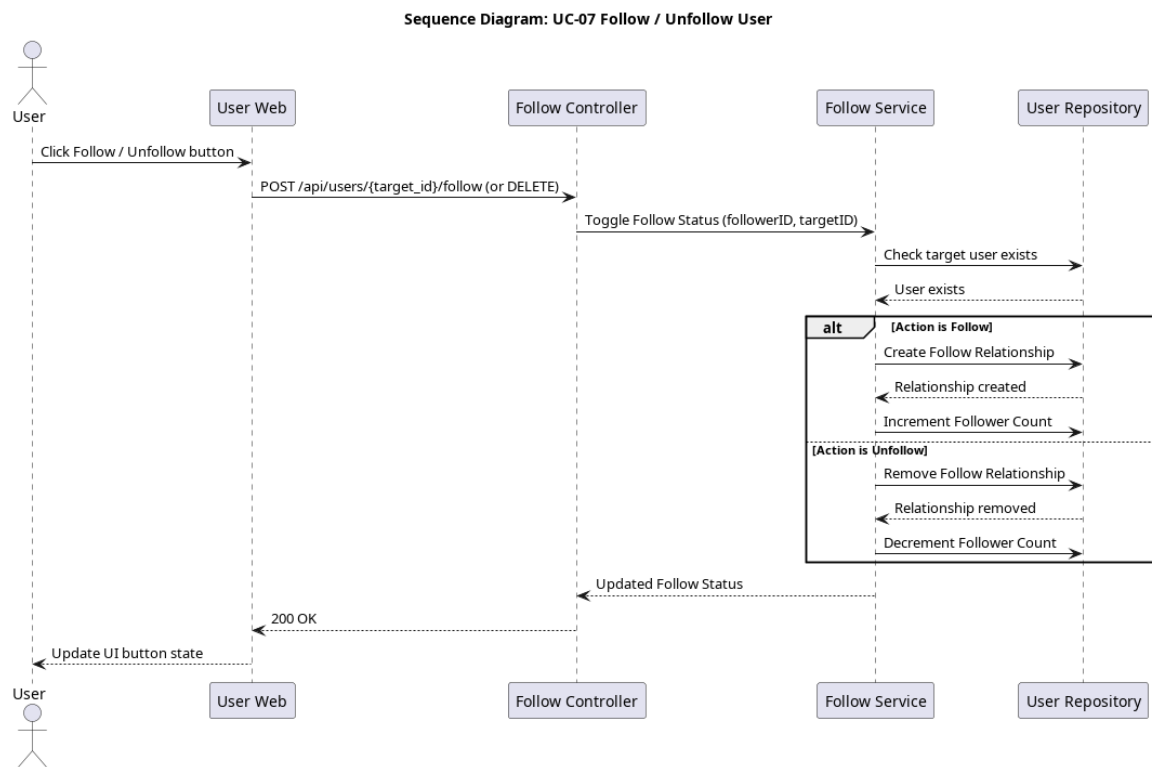


Figure 3.9: Sequence Diagram for Follow / Unfollow User

UC-08: Browse Post

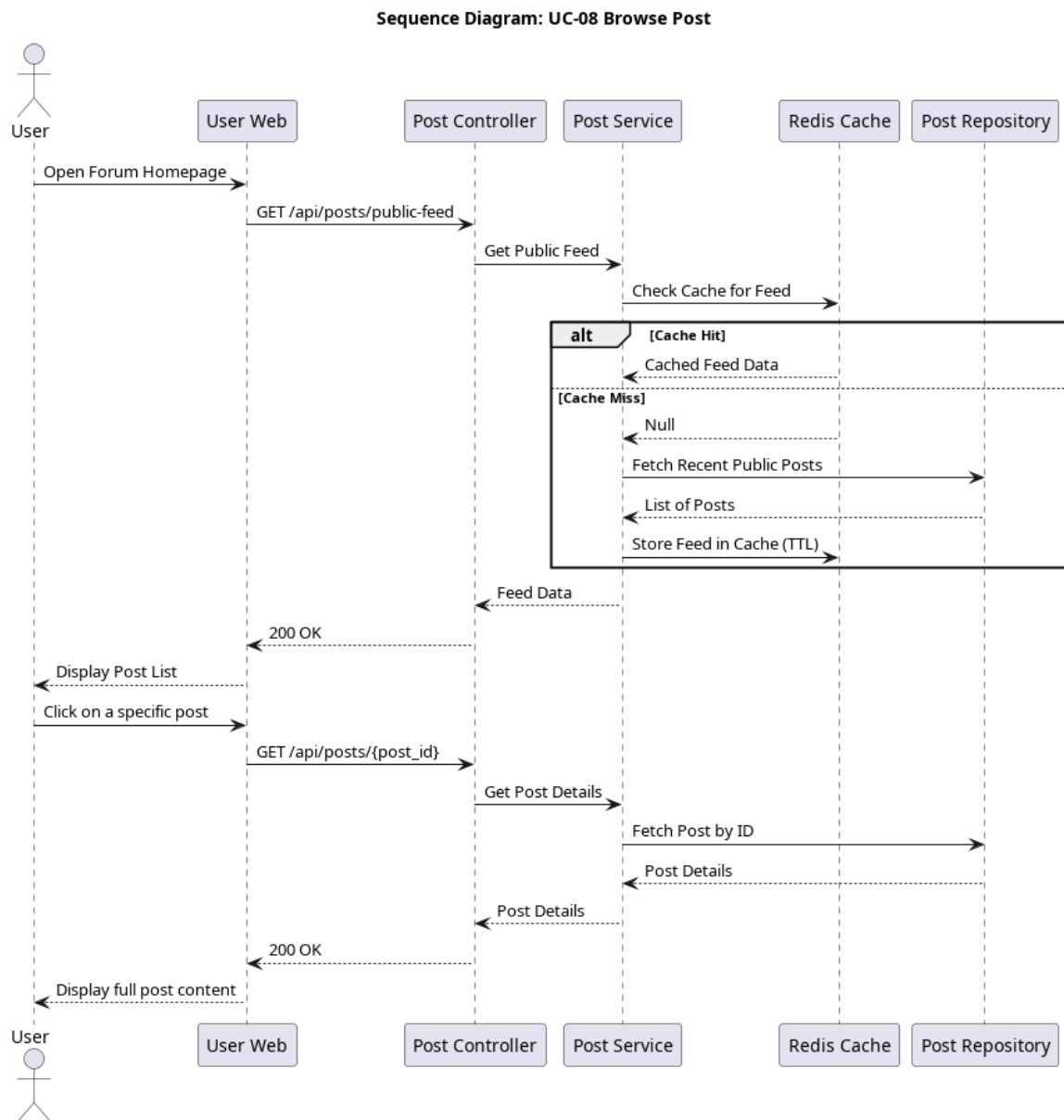


Figure 3.10: Sequence Diagram for Browse Post

UC-09: Search Content

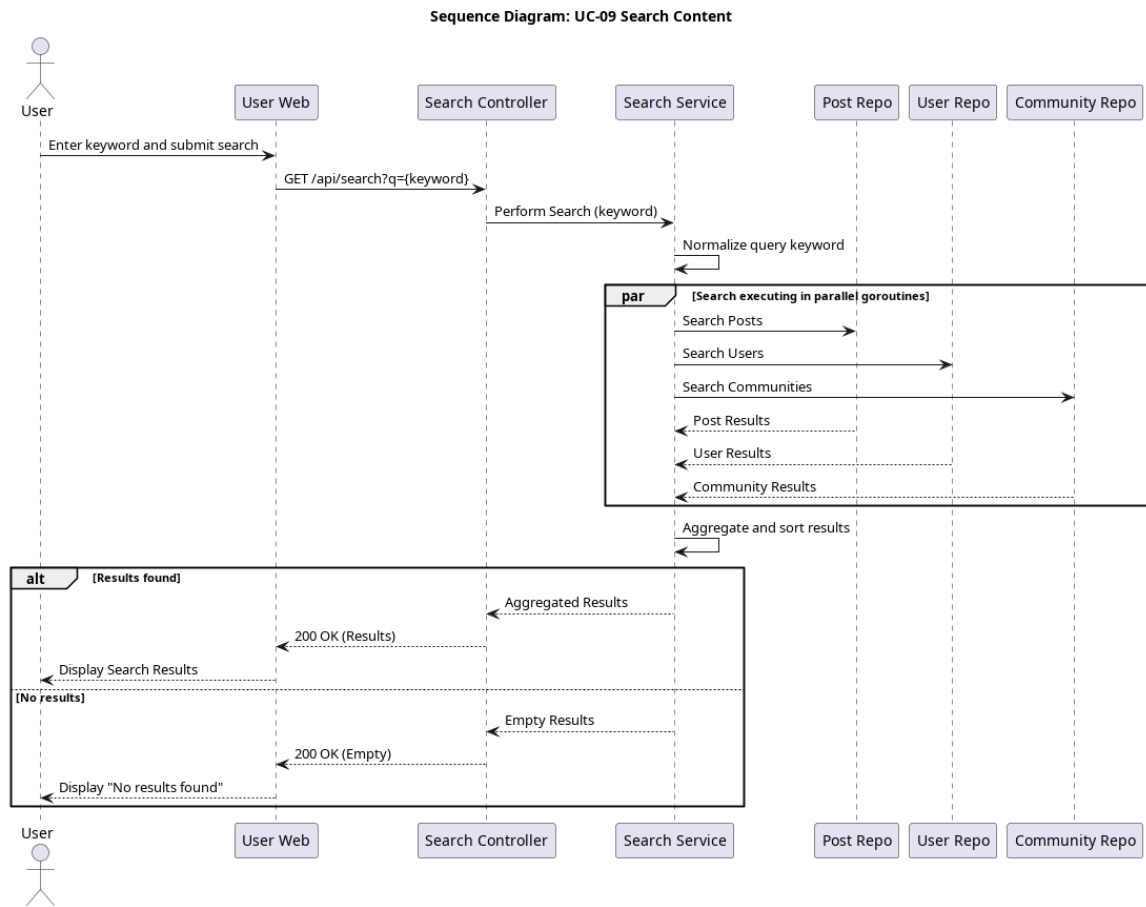


Figure 3.11: Sequence Diagram for Search Content

UC-10: Create Post

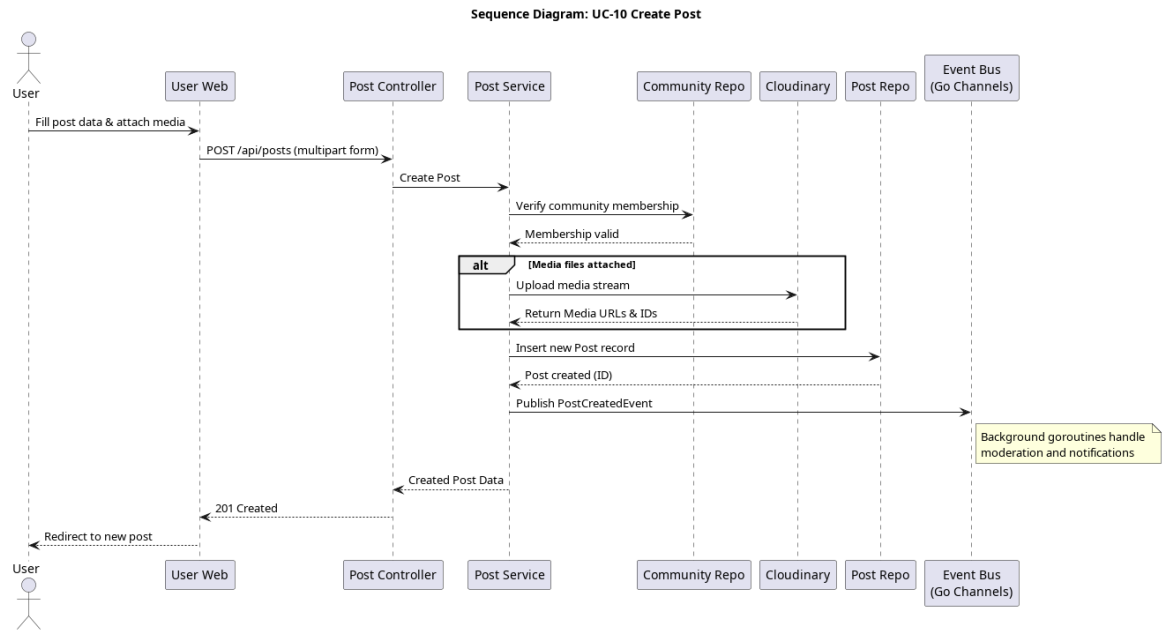


Figure 3.12: Sequence Diagram for Create Post

UC-11: Edit Post

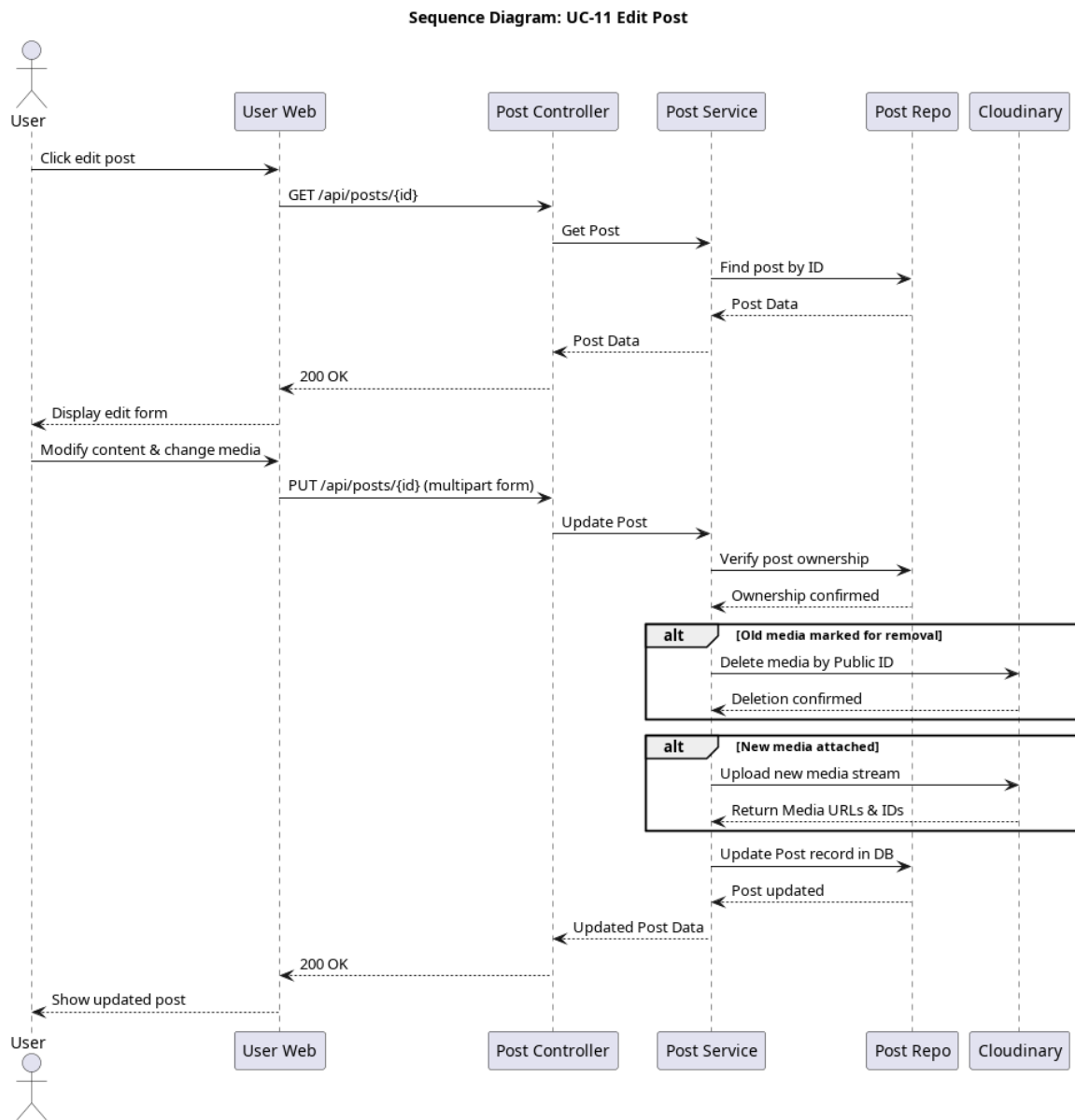


Figure 3.13: Sequence Diagram for Edit Post

UC-12: Delete Post

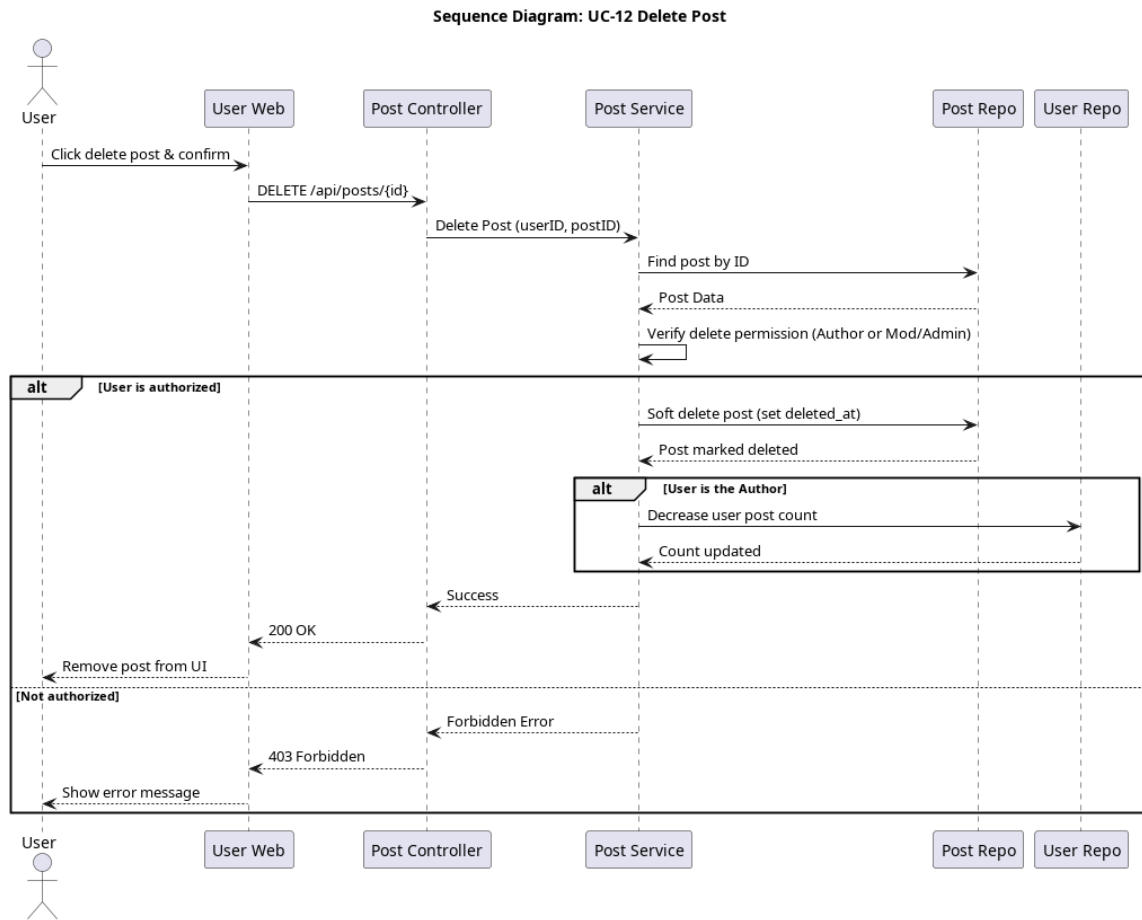


Figure 3.14: Sequence Diagram for Delete Post

UC-13: Post Comment

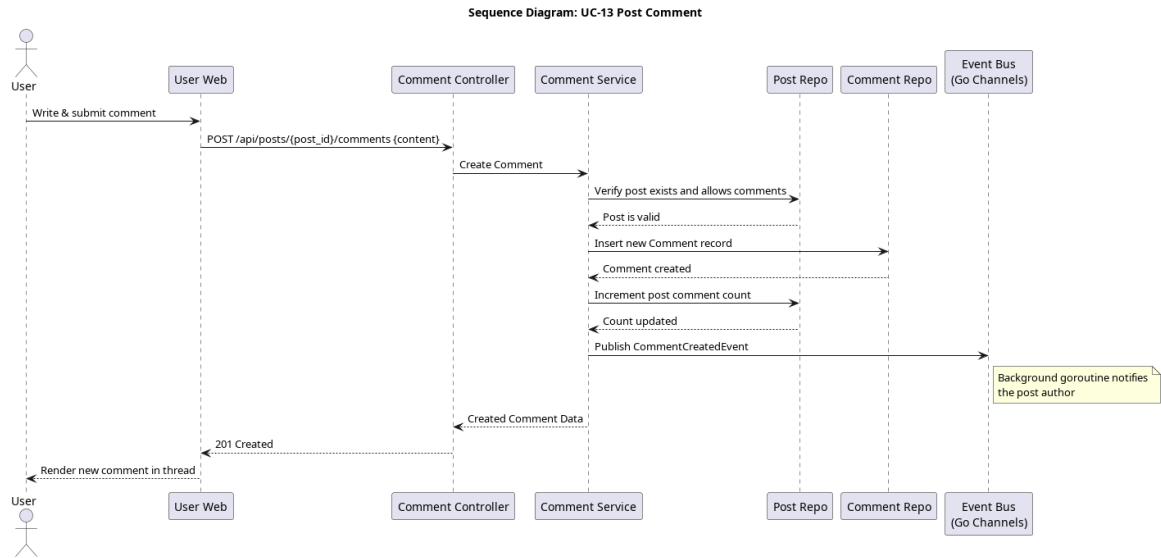


Figure 3.15: Sequence Diagram for Post Comment

UC-14: Reply to Comment

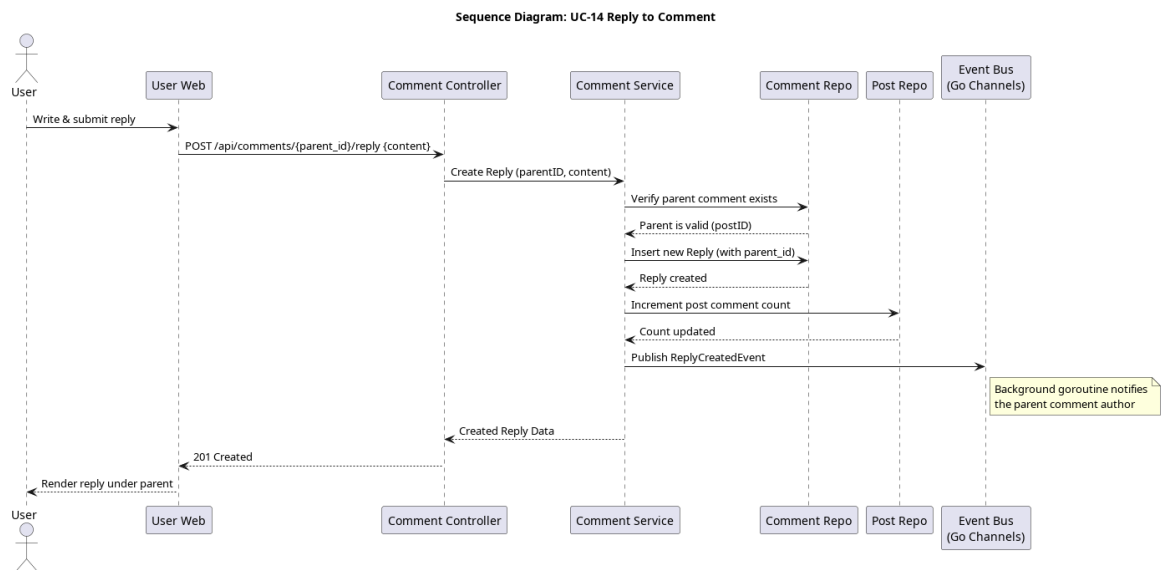


Figure 3.16: Sequence Diagram for Reply to Comment

UC-15: Vote on Content

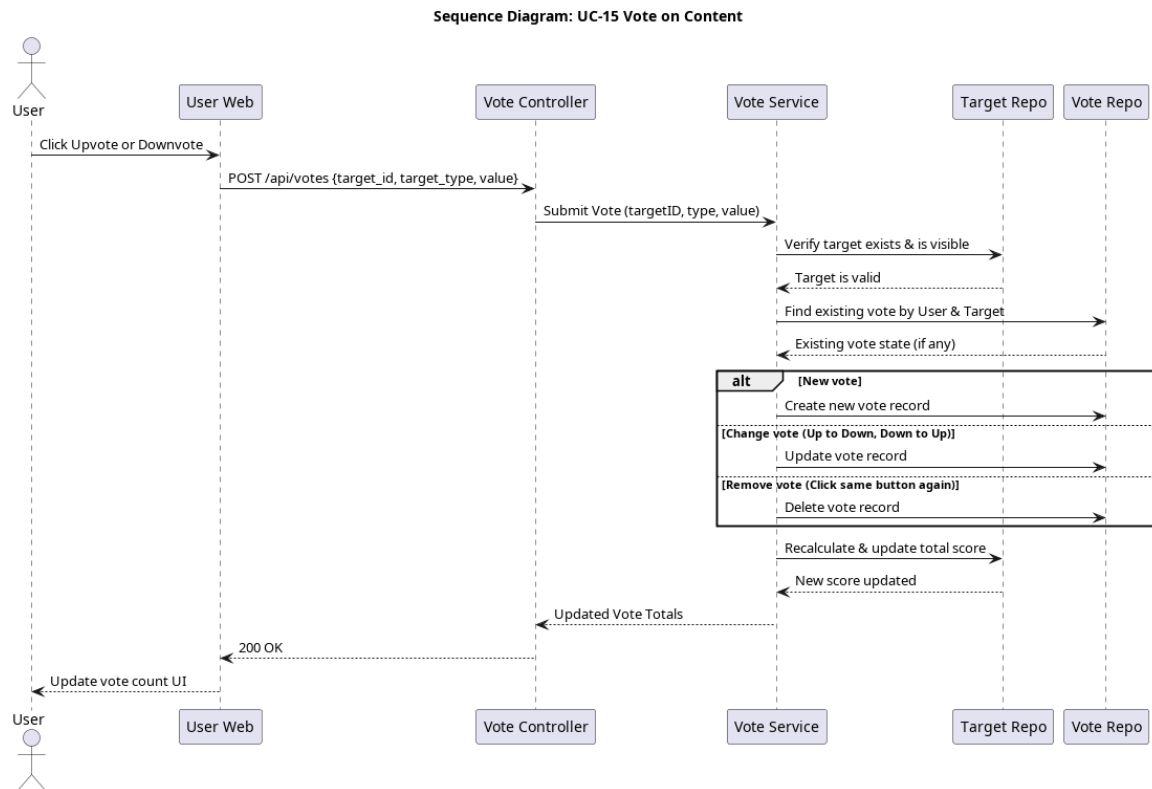


Figure 3.17: Sequence Diagram for Vote on Content

UC-16: Participate in Poll

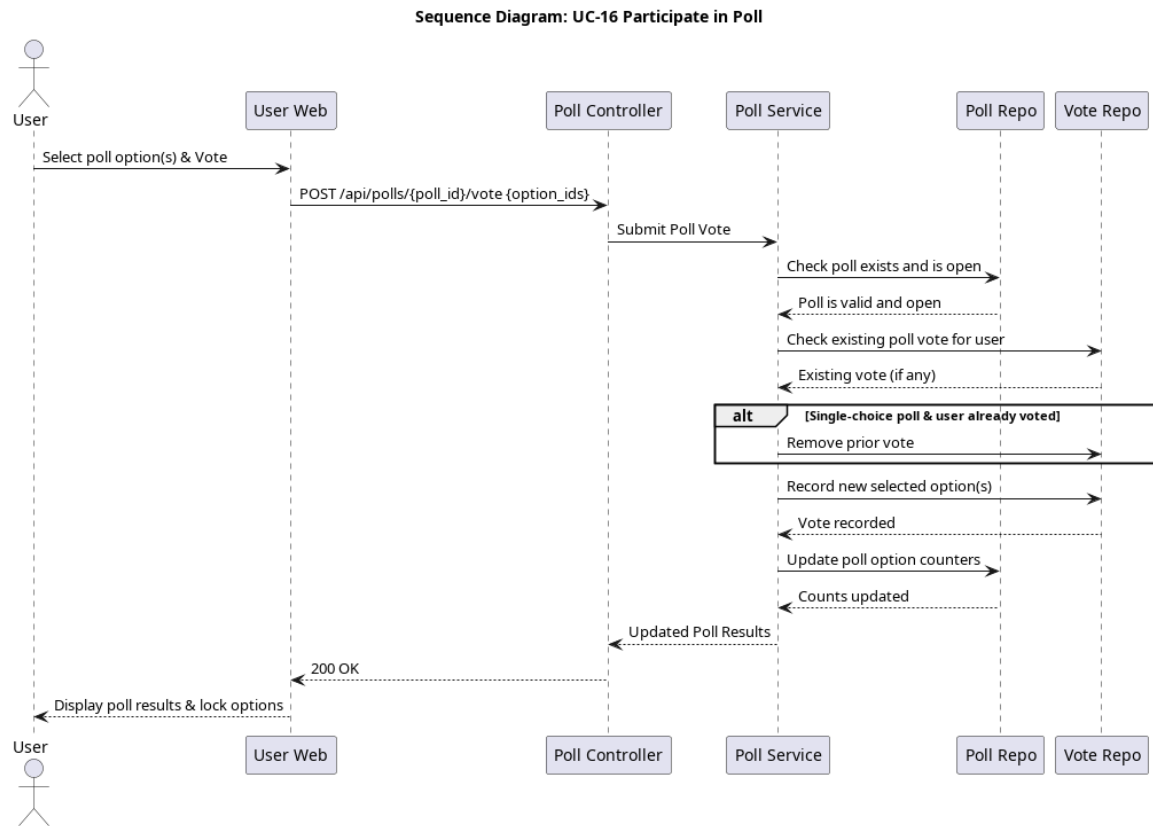


Figure 3.18: Sequence Diagram for Participate in Poll

UC-17: Save Post

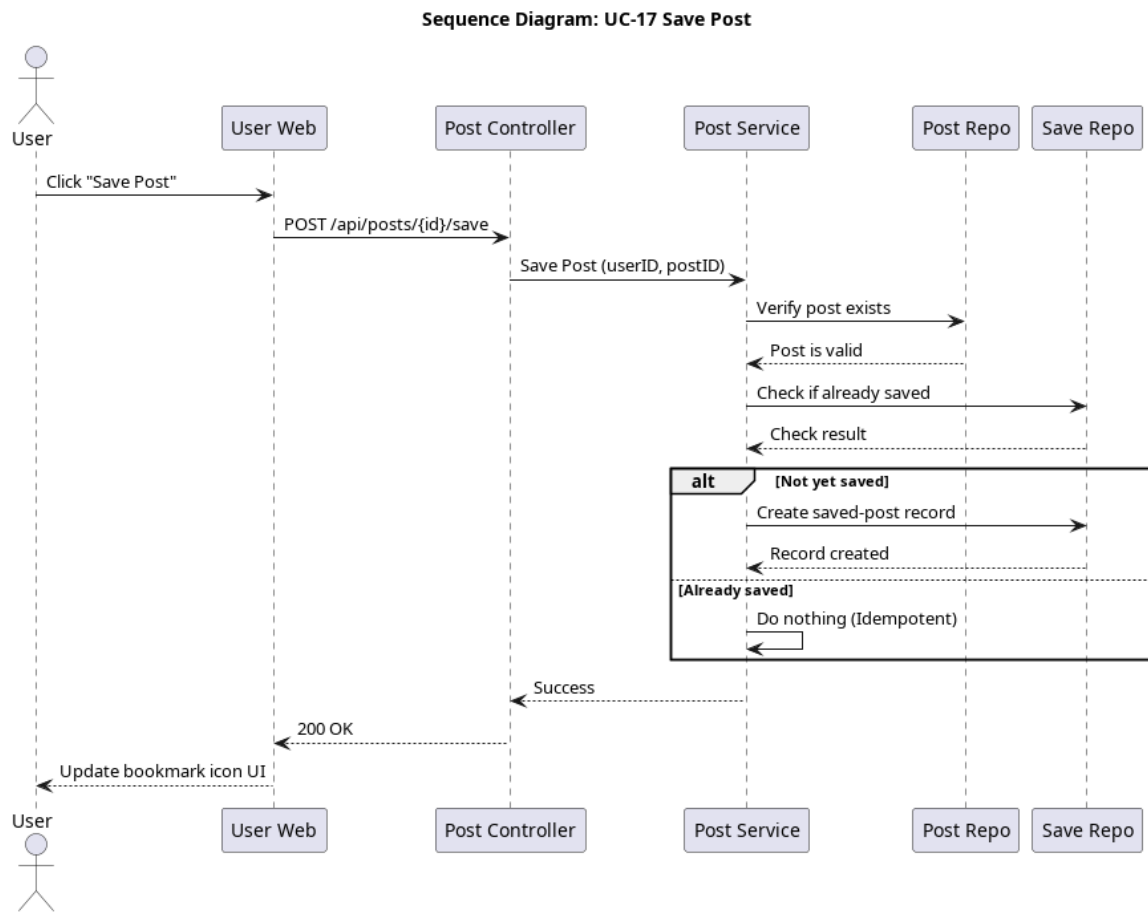


Figure 3.19: Sequence Diagram for Save Post

UC-18: Hide Post

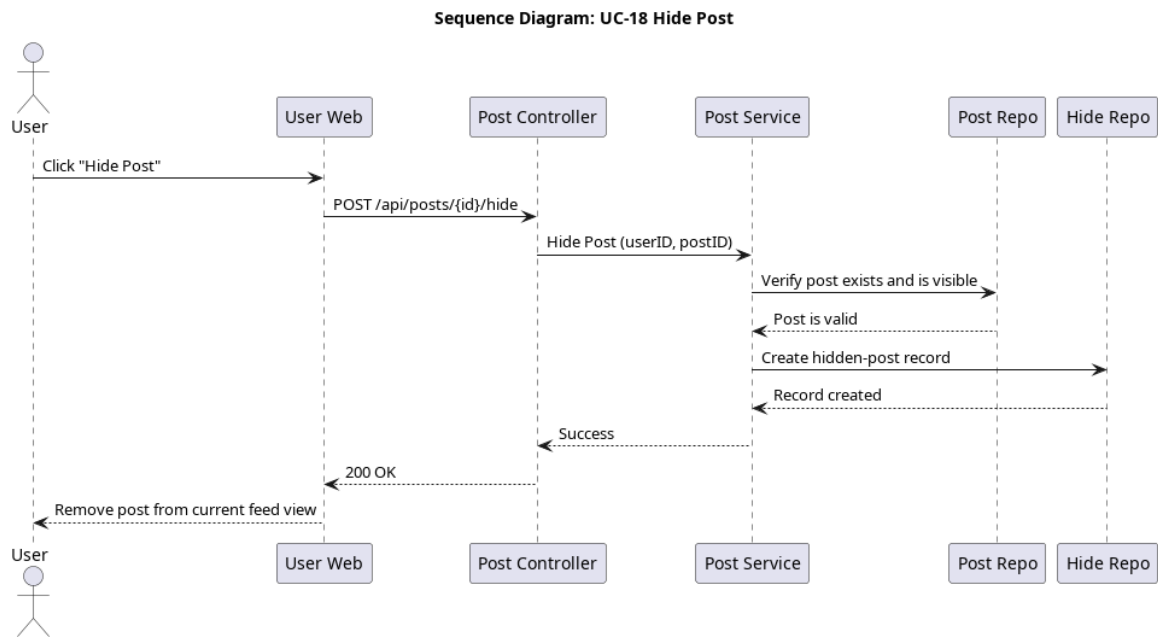


Figure 3.20: Sequence Diagram for Hide Post

UC-19: Manage Drafts

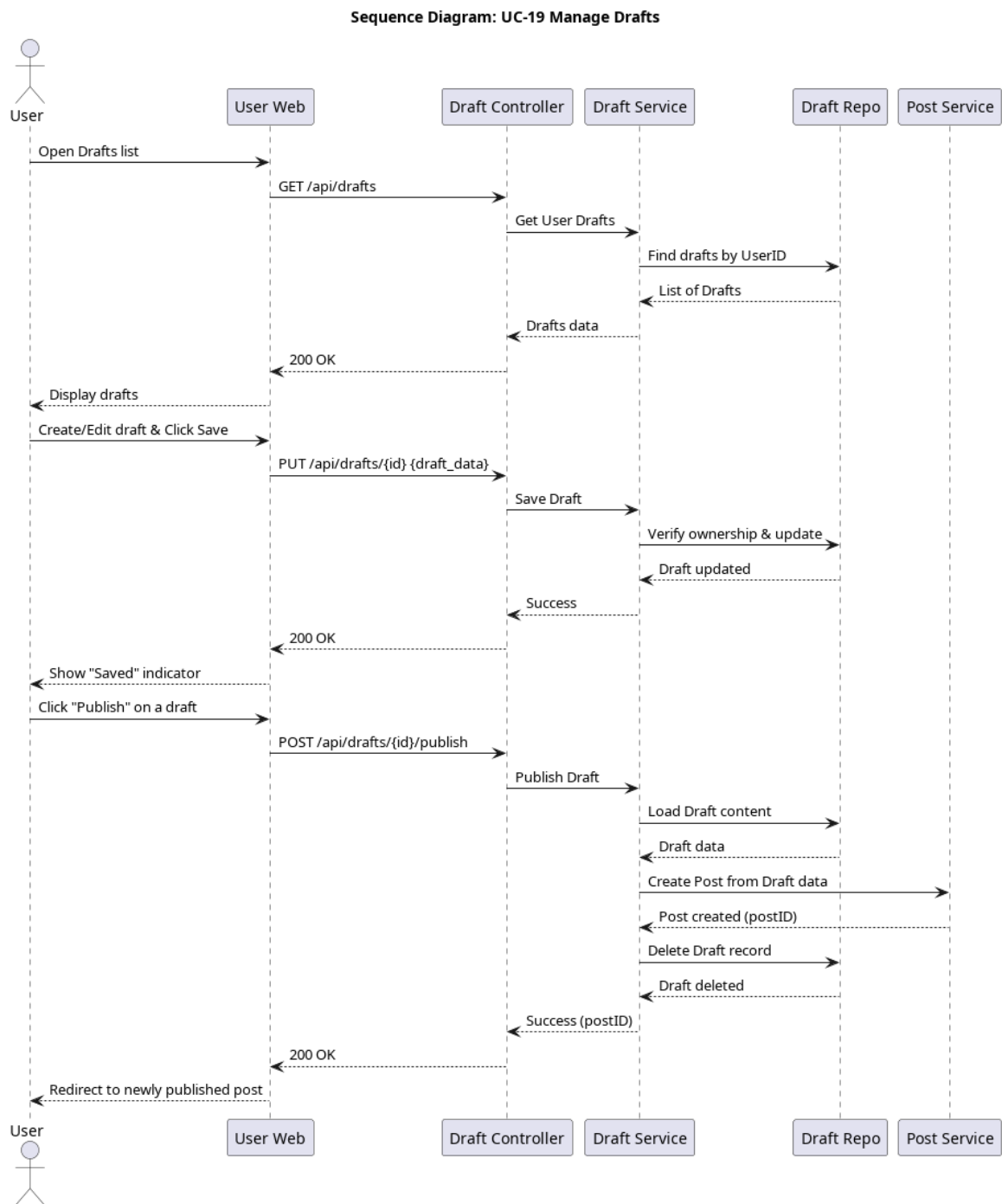


Figure 3.21: Sequence Diagram for Manage Drafts

UC-20: Private Messaging

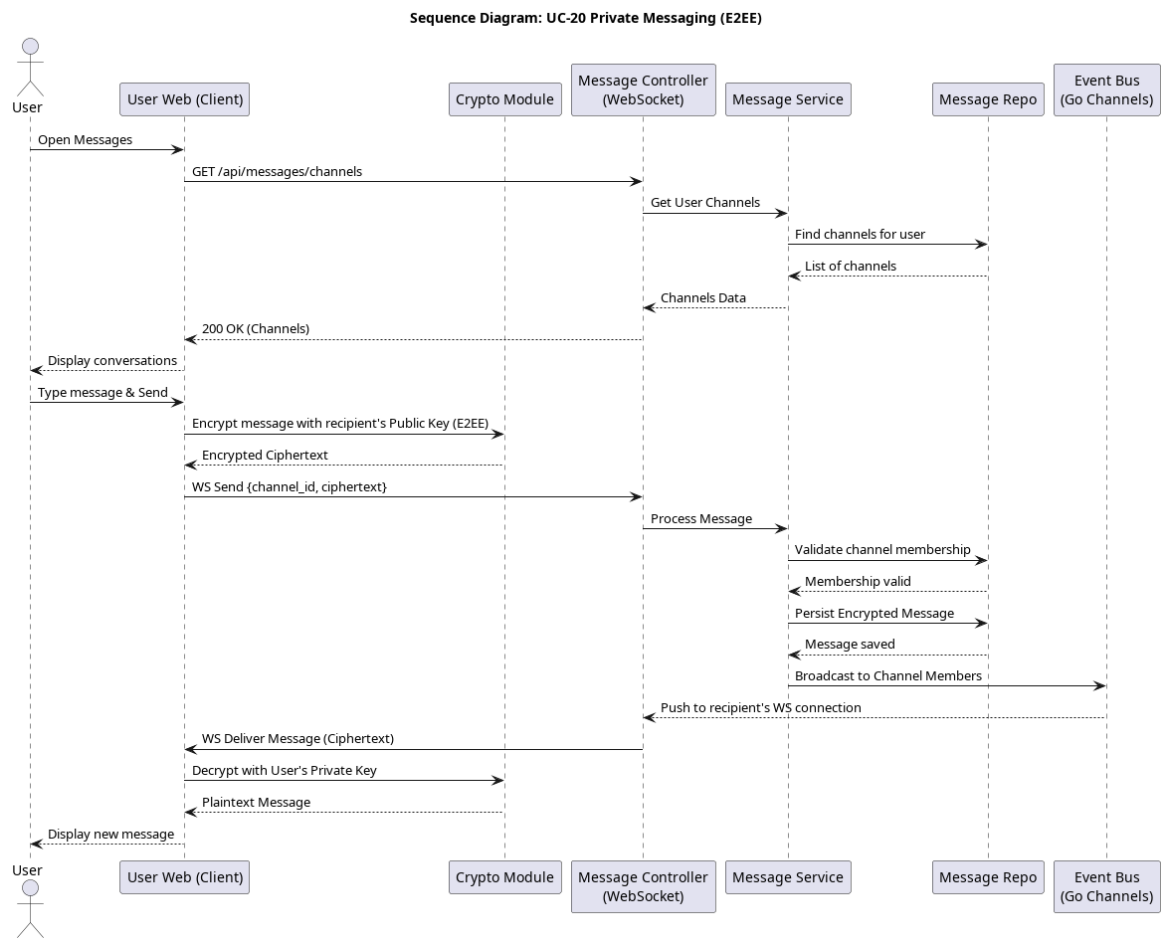


Figure 3.22: Sequence Diagram for Private Messaging

UC-21: Manage Notifications

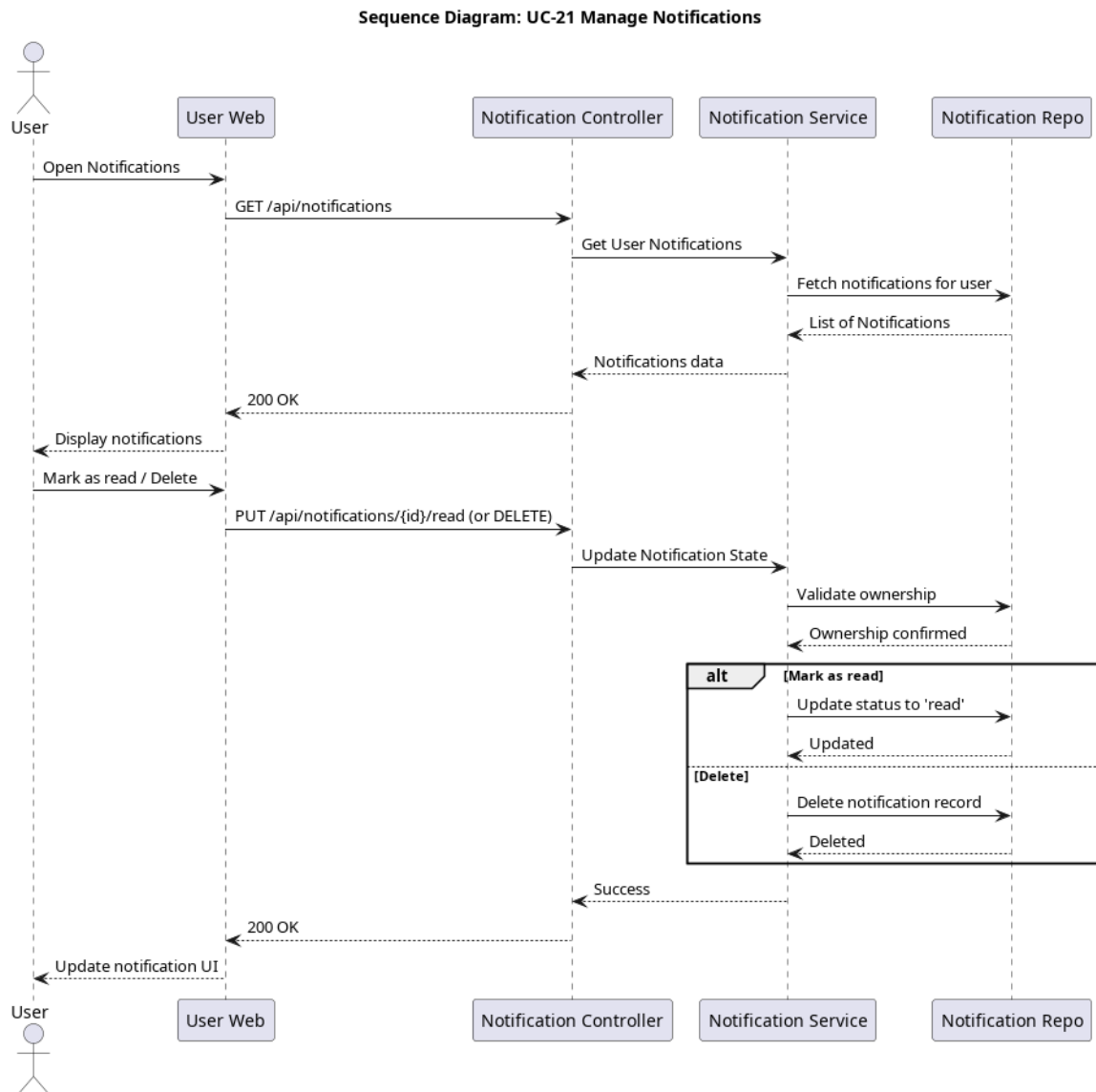


Figure 3.23: Sequence Diagram for Manage Notifications

UC-22: Report Violation

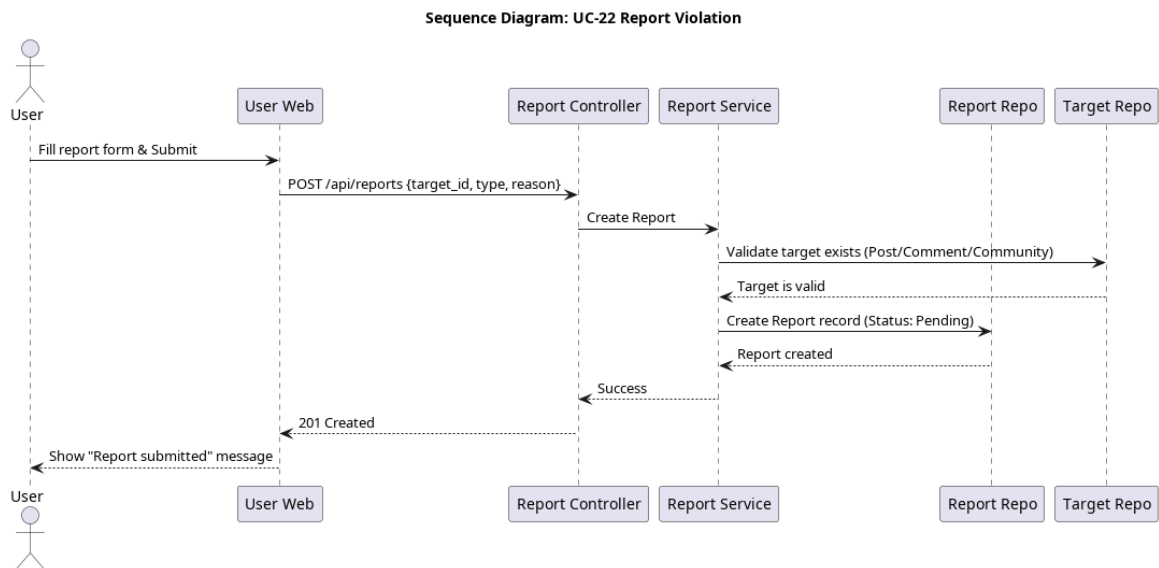


Figure 3.24: Sequence Diagram for Report Violation

UC-23: Create Community

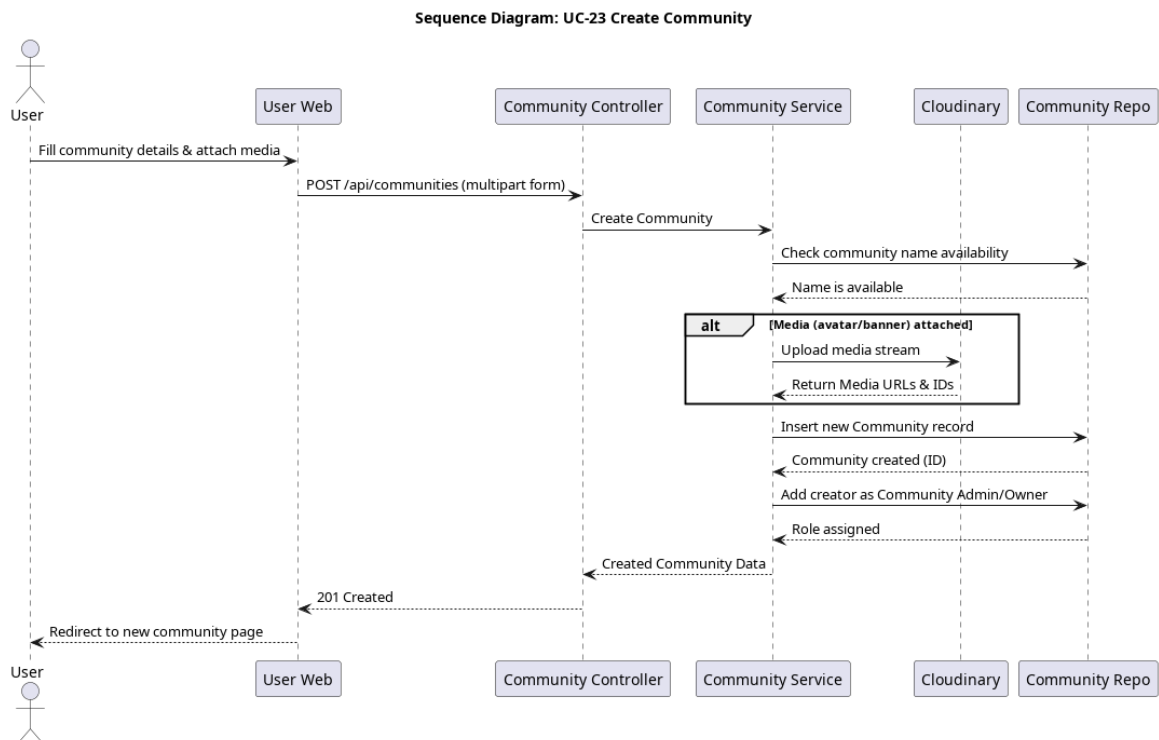


Figure 3.25: Sequence Diagram for Create Community

UC-24: Approve / Reject Post

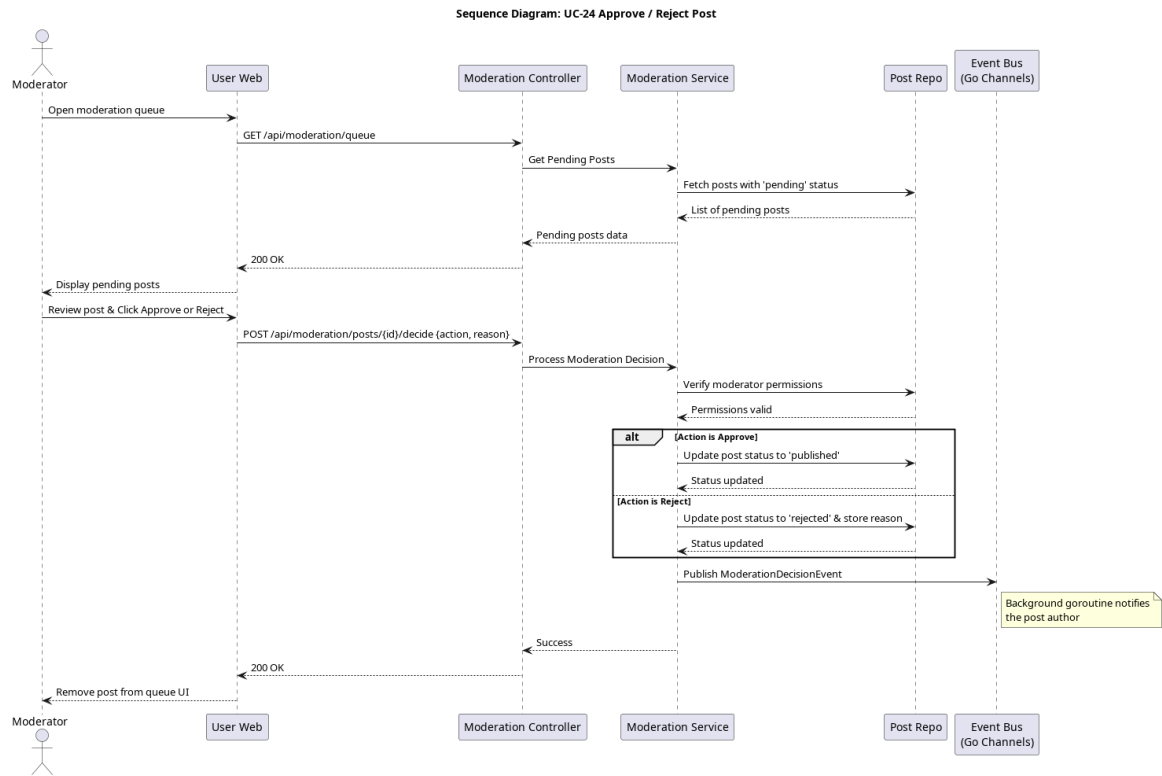


Figure 3.26: Sequence Diagram for Approve / Reject Post

UC-25: Ban / Mute User

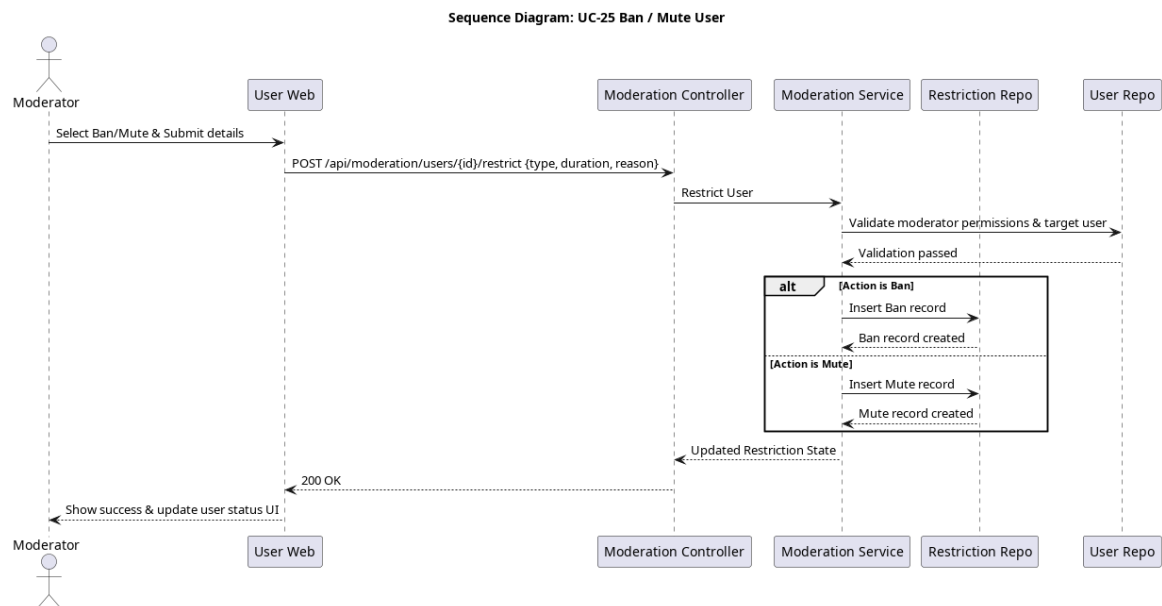


Figure 3.27: Sequence Diagram for Ban / Mute User

UC-26: Configure Community

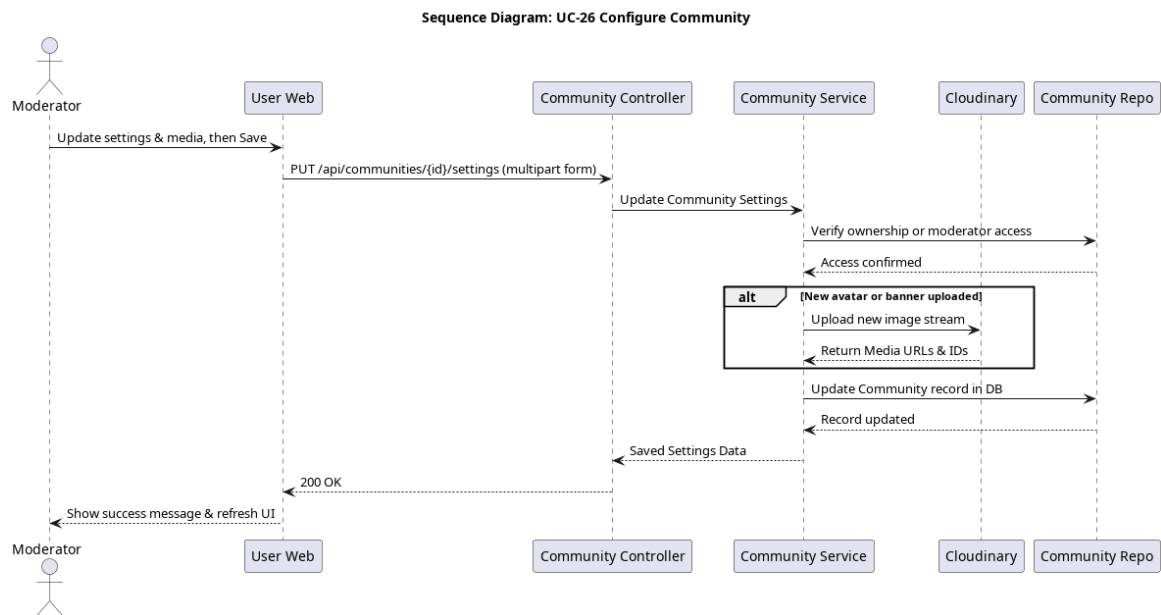


Figure 3.28: Sequence Diagram for Configure Community

UC-27: Assign Roles

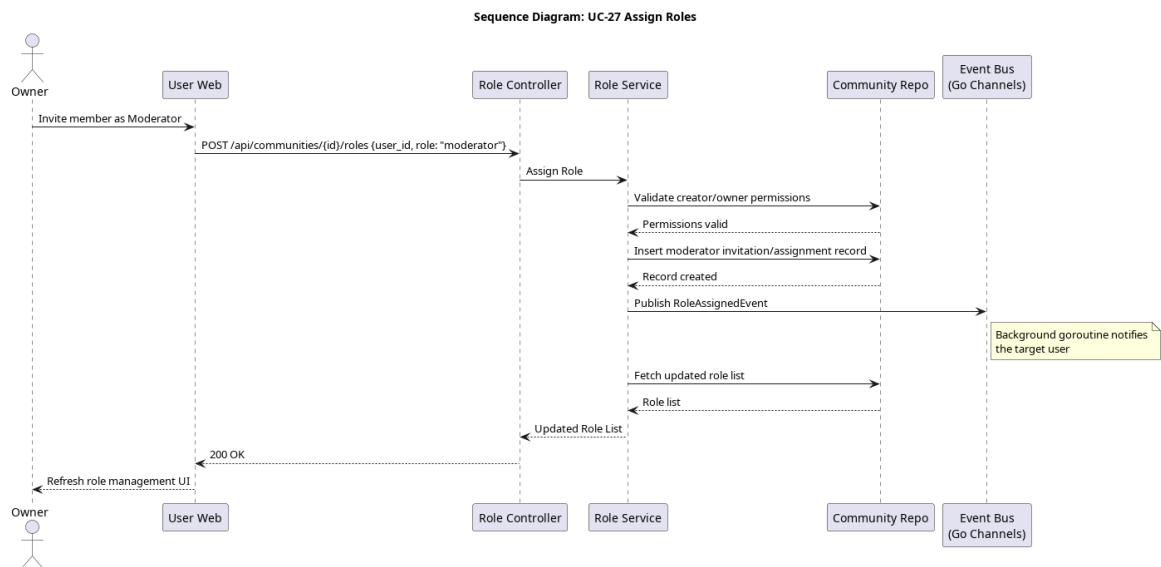


Figure 3.29: Sequence Diagram for Assign Roles

UC-28: Handle Global Reports

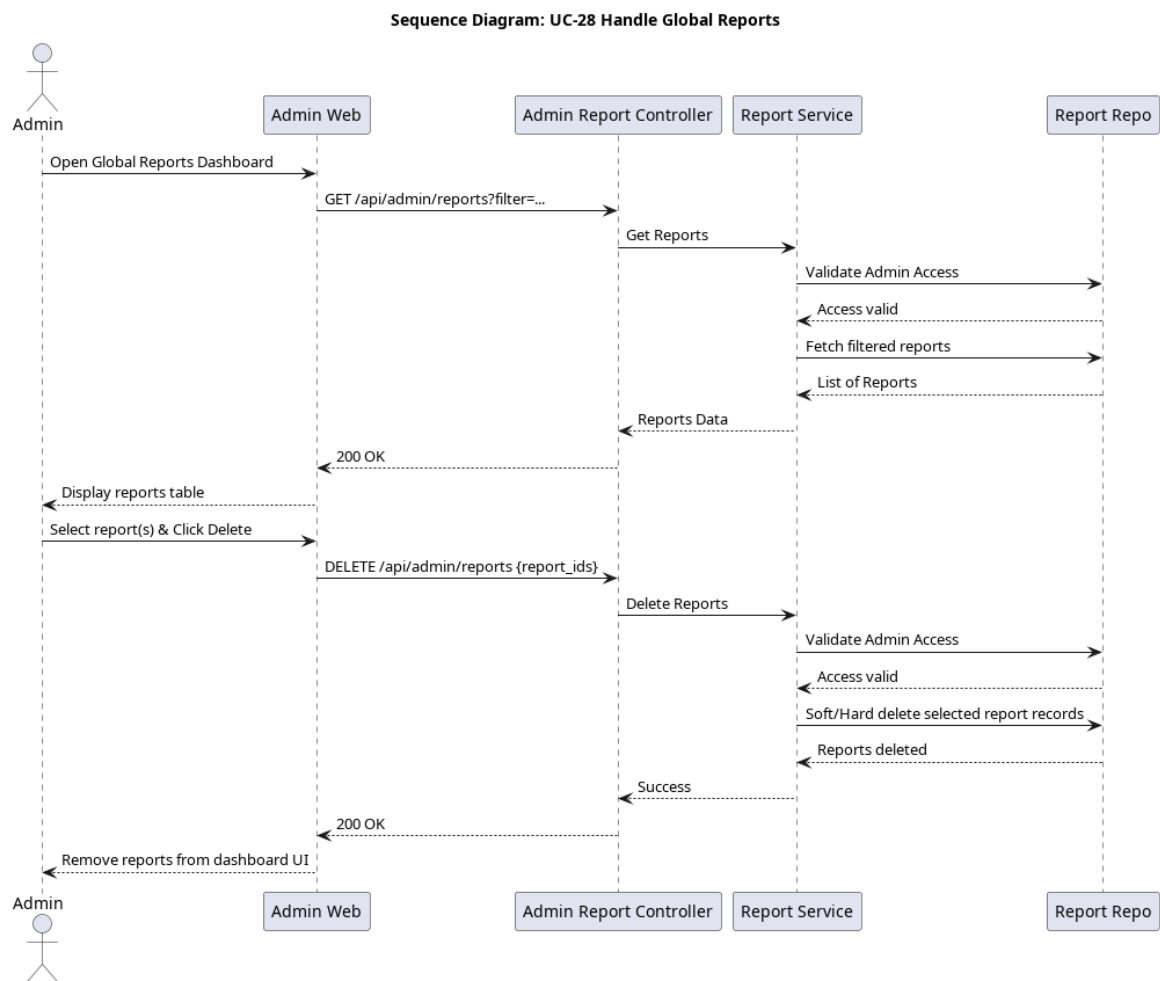


Figure 3.30: Sequence Diagram for Handle Global Reports

UC-29: Manage Platform

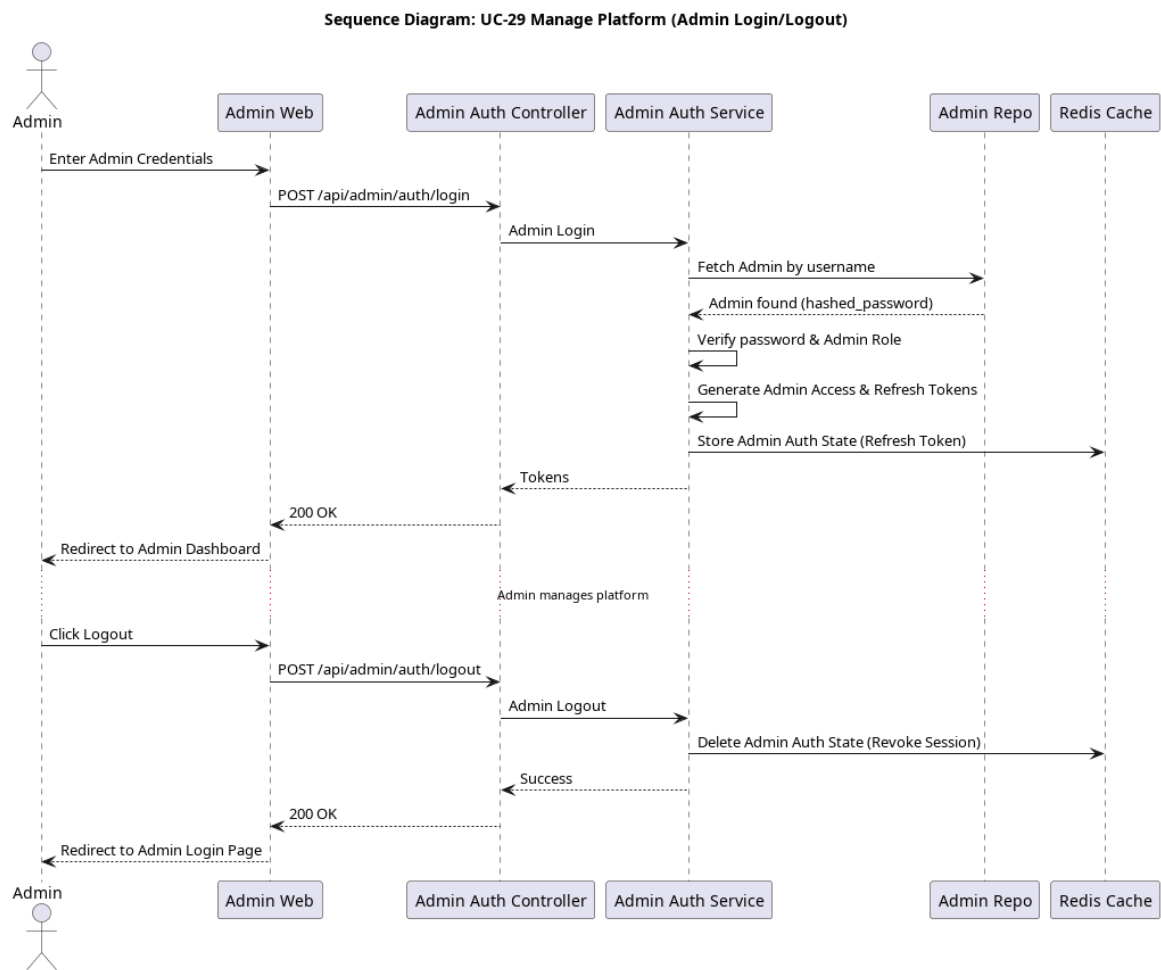


Figure 3.31: Sequence Diagram for Manage Platform

UC-30: View Analytics

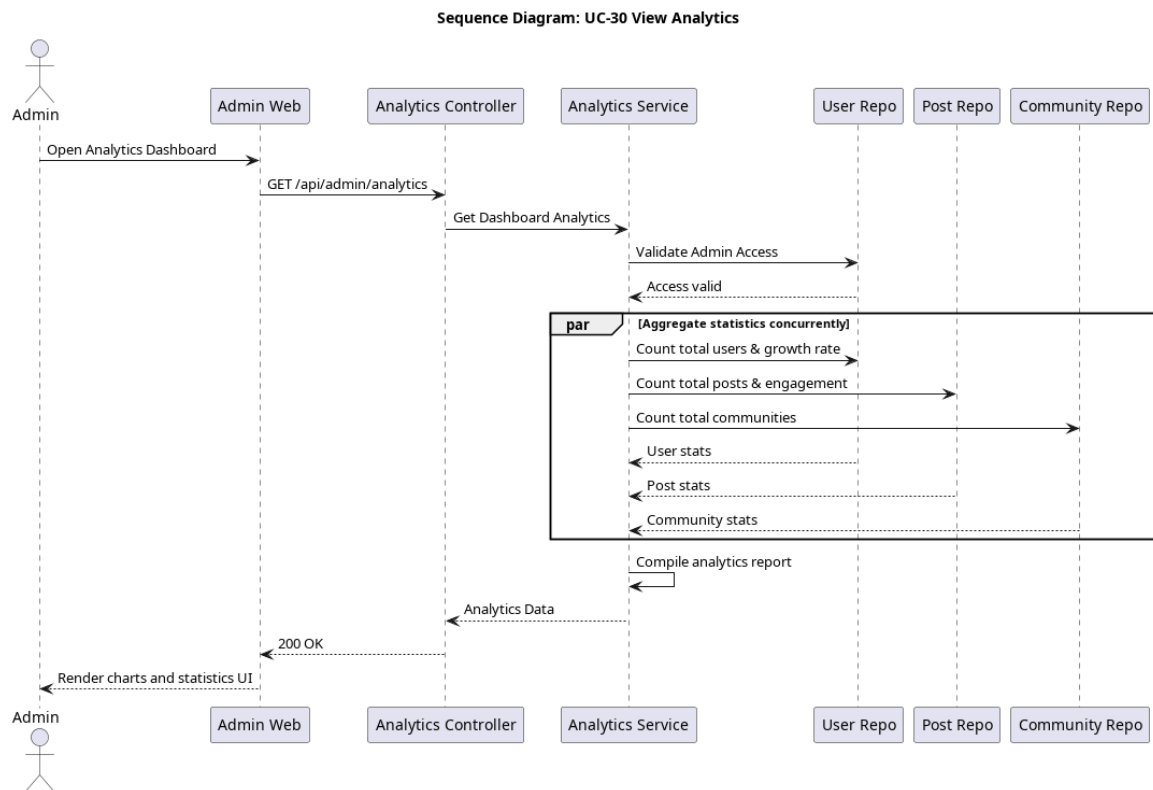


Figure 3.32: Sequence Diagram for View Analytics

3.2 Module View (Implementation Structure)

This view defines how the codebase is structured logically, mapping to the **Component Diagram** in C4.

3.2.1 Primary Presentation

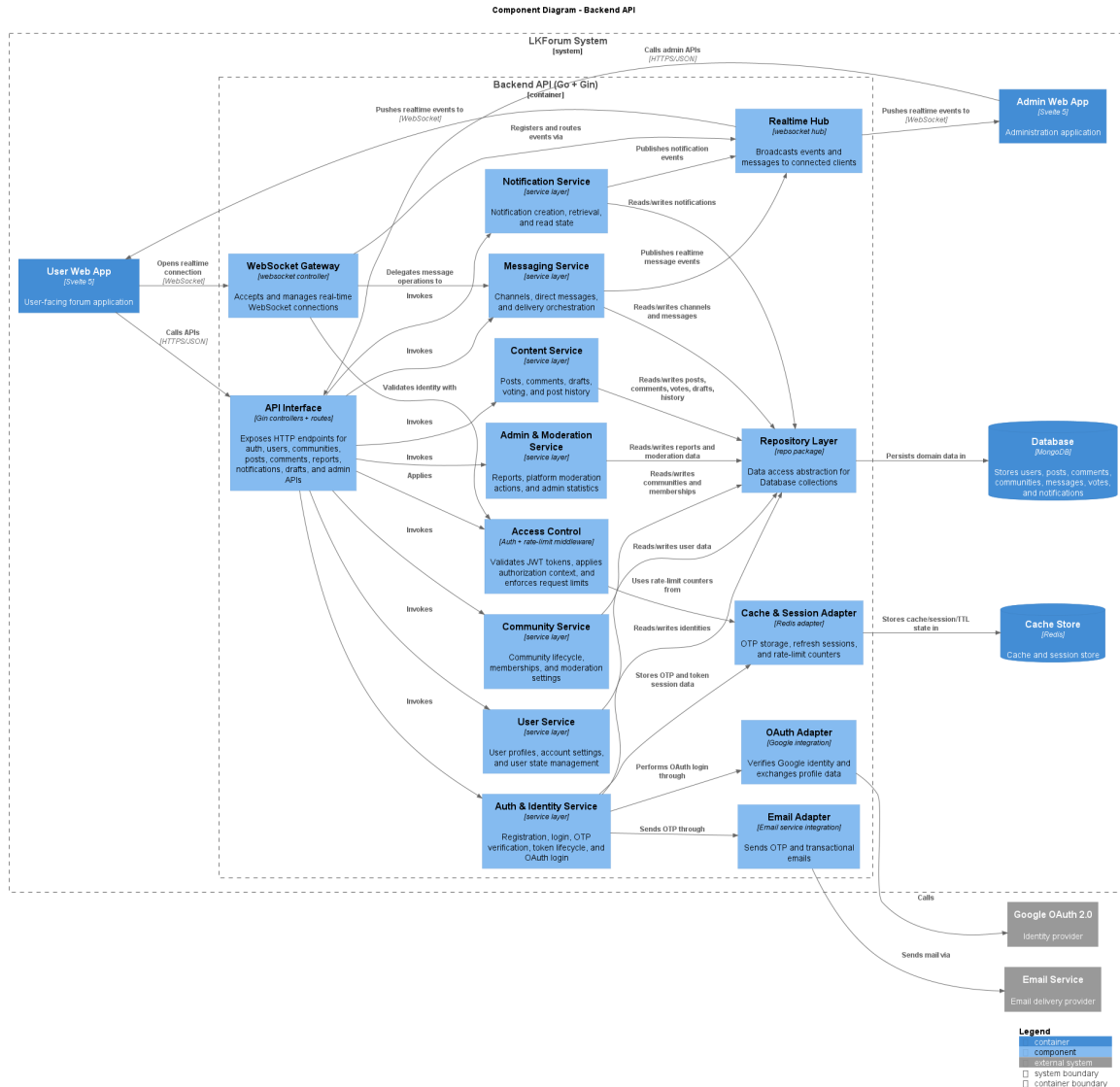


Figure 3.33: Backend Component Diagram

3.2.2 Element Catalog

- **Routes & Middlewares:** The entry point of the Go backend. It registers API endpoints and attaches global middlewares (such as JWT Validation, CORS handling, and structured Logging) before passing requests downstream.
- **Controllers (Handlers):** Responsible for parsing incoming HTTP requests (JSON/Multipart), validating input formats, and extracting parameters. Controllers do not contain business logic; they immediately delegate execution to the Service layer and format the HTTP responses.
- **Services (Business Logic):** The core of the application. Services contain all domain rules (e.g., verifying karma points, validating ownership, checking moderation states). They coordinate between Repositories, external systems, and the Event Bus.
- **Repositories (Data Access):** These modules abstract the database driver. They are respon-

sible for constructing and executing MongoDB queries (BSON) and Redis commands, returning structured Go models to the Services.

- **Event Bus (Go Channels):** An internal communication mechanism. Services can publish events (e.g., `PostCreatedEvent`) to channels. Dedicated background goroutines listen to these channels to process asynchronous tasks like sending notifications, ensuring the main HTTP request is not blocked.

3.3 Allocation View (Deployment)

This view defines the mapping of software elements to the execution environments, aligning with the **Container Diagram** in C4.

3.3.1 Primary Presentation

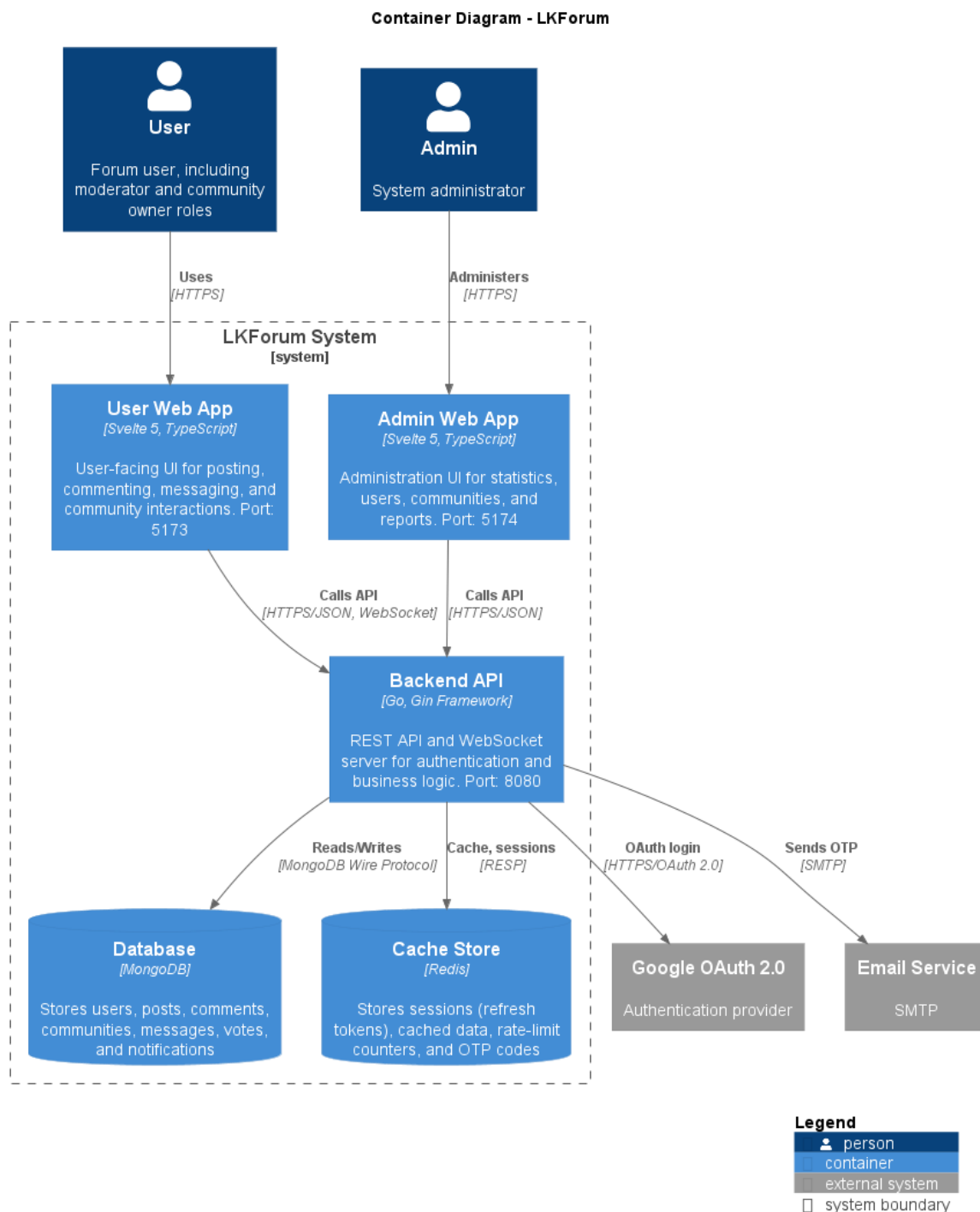


Figure 3.34: Container Diagram

3.3.2 Element Catalog

- **Docker Compose (Environment):** Acts as the deployment orchestrator for a single-node setup. It defines the internal bridged network, container dependencies, and volume mounts.

- **User Web SPA Container:** A Node.js container serving the Svelte frontend application tailored for end-users. It communicates purely via REST APIs over HTTPS.
- **Admin Web SPA Container:** A separate Node.js container serving the isolated administrative dashboard. Physical separation ensures that administrative code is not delivered to standard users.
- **Go Backend API Container:** The compiled Go binary running inside a minimal Alpine or Scratch Linux image. It exposes port 8080 and handles all business transactions and WebSocket connections.
- **MongoDB Container:** The primary persistent data store running the NoSQL database engine. Data is stored in named Docker volumes to survive container restarts.
- **Redis Container:** The in-memory data structure store used for fast-expiring tokens, OTPs, and query result caching to reduce load on MongoDB.
- **Prometheus Container:** A time-series database configured to periodically scrape the `/metrics` endpoint of the Go Backend to record system health over time.
- **Grafana Container:** The observability dashboard. It connects to Prometheus to visualize system metrics and logs in real-time.

4 Relations Among Views

The views presented above are tightly interrelated.

- The external actors defined in the **System Context View** are the direct sources of the interactions detailed in the **Sequence Diagrams**.
- The structural layers described in the **Module View** (Controllers, Services, Repositories) map exactly to the participants (lifelines) executing the logic within each Sequence Diagram.
- Finally, the execution of the compiled codebase from the Module View requires the physical infrastructure defined in the **Allocation View**, where the Go codebase becomes a running container processing the requests originating from the SPA containers.